

FINANCIAL ECONOMETRICS

MODULE 5 | LESSON 1

ARCH MODEL

Reading Time 60 minutes

Prior Knowledge Basics of time series, ARIMA

Keywords Conditional Means, Conditional Variance, Volatility Clustering, ARCH(1) Model

In the last module, we used the ARIMA model to analyze Google stock price. One key issue we observed was that Google stock price does not exhibit constant variance throughout our modeling time period. This issue does not just happen to Google's stock price. It is very common among many financial asset returns. In this module, we are going to address this phenomenon and find a model to analyze it. We will introduce the ARCH model and GARCH model to handle this issue. They are popular methods in finance to model volatility for many asset returns. In this lesson, we will start with learning the ARCH model and its properties. We will simulate the ARCH process to give a visual example and learn how to use ACF and PACF plots to identify the ARCH process.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from arch.univariate import ARCH, ConstantMean, Normal
from scipy import stats

plt.rcParams["figure.figsize"] = (16, 9) # Figure size and width
# note that !pip install arch was previously run in this notebook to make the follo
```

```
In [2]: # Download the dataset
m5_data = pd.read_csv("M5. goog_eur_10.csv")

# Convert date variable to date format and set index
m5_data["Date2"] = pd.to_datetime(m5_data["Date"], format="%m/%d/%Y")
goog = m5_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
```

1. Common Features of Financial Asset Returns

There are several special features from financial asset returns. The existence of these features motivates us to use ARCH and GARCH models to analyze financial asset returns. In this section, we are going to go through these features first so that you will have a better understanding of why we need to learn the ARCH and GARCH models. The first feature we'll talk about is the reason to use asset returns to conduct a research project instead of asset prices.

1.1 Asset Return Time Series Data is More Stable Than Asset Price Time Series Data

In a lot of financial research, we usually use asset returns instead of asset prices to conduct analysis for a financial asset. Here are some reasons to use asset returns:

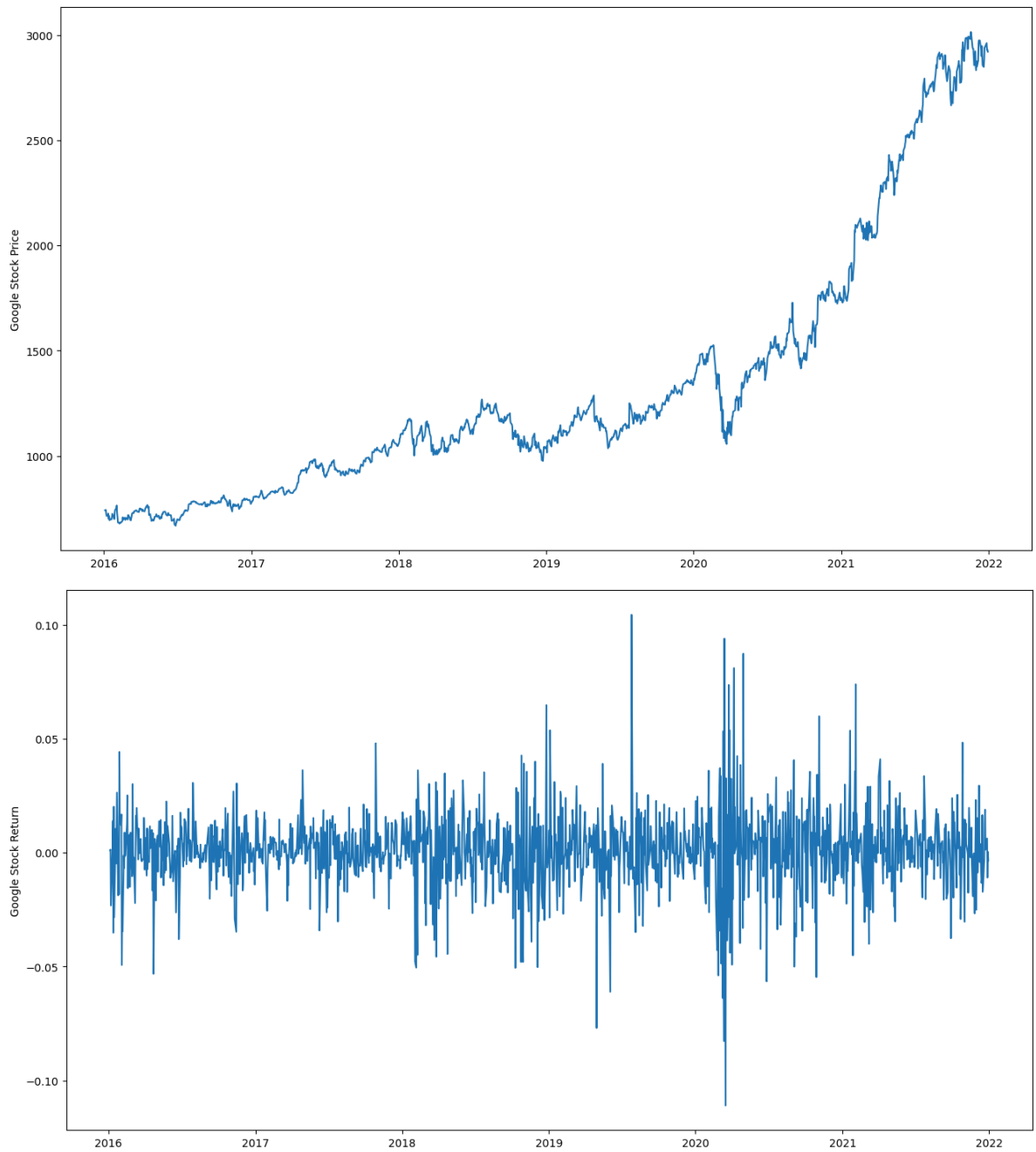
- a. An asset return provides same-scale time series data for all financial assets. It is easier for comparison among several financial assets with different price levels.
- b. Asset return time series is usually more stable than asset price time series. Let's use Google's stock price and return as an example.

Figure 1: Google Daily Stock Price and Return Historical Chart 2016 – 2021

```
In [3]: # Plot Google Daily stock price and return

# Google Daily stock price
plt.plot(goog.GOOGLE)
plt.ylabel("Google Stock Price")
plt.show()

# Google Daily stock price return
goog["GOOGLE_R"] = goog.GOOGLE.pct_change()
plt.plot(goog.GOOGLE_R)
plt.ylabel("Google Stock Return")
plt.show()
```



In figure 1, the upper chart is Google's daily stock price chart. The bottom chart is Google's stock daily return chart. We can see that Google's price plot has an upward trend whereas Google's return plot oscillates around 0 and is stable. We can say the return on average is 0.

c. If the asset return is small, we can approximate the return by using the difference of the natural log of price. Here is the formula:

$$r_t = \frac{p_t - p_{t-1}}{p_{t-1}} \approx \log(p_t) - \log(p_{t-1})$$

1.2 The Volatility of Asset Returns Can Vary During Different Time Periods

In portfolio management and risk management, managers control risk by monitoring the volatility of the underlying assets. Variance and standard deviations are common metrics to measure volatility. We can see from figure 1 that the local variance of Google stock returns changes substantially across the time period. This time series data exhibits heteroskedasticity. In the return plot, we can see there are pockets of time in the whole period where the movement of returns are more volatile than others. These high volatility pockets do not spread randomly during the whole time period. We call these **volatility clustering**. We notice that there is autocorrelation of asset returns. A highly volatile return one day would be followed by another highly volatile return the next day. It means that the past volatilities will still have an impact on today's volatility. The highly volatile periods tend to group together. This is a key property that later we would like to model.

1.3 Asset Return Distribution Has Heavier Tails Than Normal Distribution

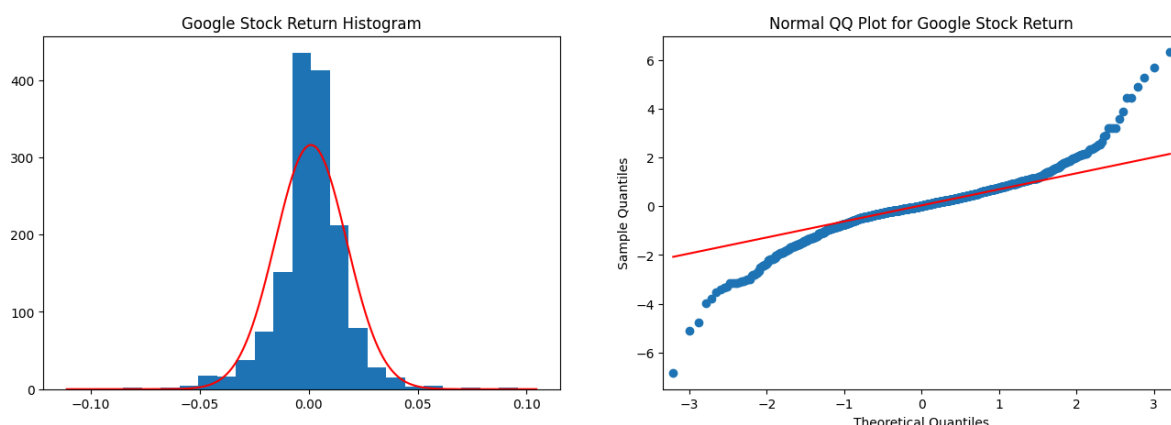
We have discussed many times in the previous lessons that financial asset returns often show a distribution with heavier tails than normal distribution. Let's take a look at Google's stock return distribution to see if the same feature shows up.

Figure 2: Histogram and Normal QQ Plot for Google Stock Returns

```
In [4]: # Histogram and Normal QQ Plot for Google Stock Returns
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
goog_r = goog.GOOGLE_R.dropna()

# Histogram with density
x = np.linspace(min(goog_r), max(goog_r), len(goog_r))
(mu, sigma) = stats.norm.fit(goog_r)
values, bins, _ = ax1.hist(goog_r, bins=25) # Histogram
ax1.plot(x, stats.norm.pdf(x, mu, sigma) * sum(values * np.diff(bins)), "r") # Den
ax1.set(title="Google Stock Return Histogram")

# Normal QQ plot
sm.qqplot(goog_r, stats.norm, fit=True, line="q", ax=ax2)
ax2.set(title="Normal QQ Plot for Google Stock Return")
plt.show()
```



In figure 2, we show Google's stock return histogram and QQ plot. Both plots demonstrate that Google's stock return exhibits fat tail characteristic as well.

2. Conditional Means and Conditional Variances

We have talked about special features for financial asset returns time series. Before we dive into models to formally analyze those features, let's start by learning some concepts that will prepare us for explaining the models later.

We talked about conditional probability in the last module. Now let's learn how to obtain conditional means and conditional variances.

Assume X and Y are two discrete random variables.

The **conditional mean** of Y given X is as follows:

$$\mu_{Y|X} = \sum f(Y|X)Y = E(Y|X)$$

where $f(Y|X)$ is the conditional probability of Y given X .

The **conditional variance** of Y given X is as follows:

$$\sigma_{Y|X}^2 = \sum Y^2 f(Y|X) - \mu_{Y|X}^2 = E(Y^2|X) - \mu_{Y|X}^2$$

We can see that the calculation for conditional means and conditional variances are very similar to calculation for regular means and variances. The difference is we use conditional probability to calculate conditional means and conditional variances.

Why do we need to know conditional means and conditional variances in the time series context? The central idea is how we can incorporate the concept of using past value information to predict today's value in a model. We actually did that for the ARMA model in the last module. We used the concept of conditional mean to run an ARMA model. Let's use the following example to illustrate the idea.

Consider an AR(p) model with a constant conditional variance

$Var(X_t|X_{t-1}, X_{t-2}, \dots, X_{t-p}) = \sigma^2$. We can write the AR(p) model as follows:

$$X_t = f(X_{t-1}, X_{t-2}, \dots, X_{t-p}) + e_t$$

where white noise e_t has mean = 0 and constant conditional variance σ^2 , $f()$ is the conditional mean for X_t given $X_{t-1}, X_{t-2}, \dots, X_{t-p}$ and σ^2 is the constant conditional variance for X_t given $X_{t-1}, X_{t-2}, \dots, X_{t-p}$.

Now if we want to introduce non-constant variance like we observed from the Google data in the last section to the above AR(p) model, we can rewrite the model as:

$$X_t = f(X_{t-1}, X_{t-2}, \dots, X_{t-p}) + \sigma(X_{t-1}, X_{t-2}, \dots, X_{t-p}) e_t$$

Where white noise e_t has conditional mean = 0 and conditional variance = 1, $\sigma(X_{t-1}, X_{t-2}, \dots, X_{t-p})$ is the standard deviation for X_t given $X_{t-1}, X_{t-2}, \dots, X_{t-p}$. This standard deviation will change when $X_{t-1}, X_{t-2}, \dots, X_{t-p}$ change. Since $\sigma(X_{t-1}, X_{t-2}, \dots, X_{t-p})$ is standard deviation, it should be non-negative. Later, we will provide regularity conditions to make sure this function is non-negative.

What we just described is a general variance function model for conditional variance. ARCH and GARCH are all types of variance function models.

In the following sections, we are going to focus our analysis on asset returns. The common observation is that the conditional mean of asset return is 0, so we will drop the conditional mean part from the above model to focus on the conditional variance part in the following sections. We will start with a simple ARCH(1) model in this lesson. Then, we will introduce a general ARCH(m) model. In the next lesson, we will expand the ARCH model and introduce the GARCH model.

3. ARCH(1) Model

Now let's formally introduce our first conditional variance model, ARCH(1) model. ARCH stands for autoregressive conditional heteroskedasticity. If we have a stationary asset return time series r_t , we can define ARCH(1) model for r_t as follows:

$$r_t = \sigma_t e_t$$

Where $e_t \sim iidN(0, 1)$ and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2$$

with $\alpha_0 > 0$ and $0 \leq \alpha_1 < 1$.

The design of the above variance equation makes sure the variance is non-negative. The requirement of $\alpha_1 < 1$ also ensures r_t is stationary with finite variance. Also in many cases, instead of assuming e_t to be standard normal distribution, we can assume e_t follows a

Student's t-distribution. Lastly, the 1 in ARCH(1) refers to the lag 1 asset return in the variance equation.

Now let's review the properties of ARCH(1) model/process.

3.1 ARCH(1) is Strictly Stationary

From $r_t = \sigma_t e_t$ and $\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2$, we know

$$r_t^2 = \alpha_0 e_t^2 + \alpha_1 e_t^2 r_{t-1}^2$$

By iterating the above definition, we can show that

$$r_t = e_t \sqrt{\alpha_0 \left(1 + \sum_{j=1}^{\infty} \alpha_1^j e_{t-1}^2 \cdots e_{t-j}^2 \right)}$$

We will leave the proof as one open-ended question.

3.2 r_t is Conditionally Normally Distributed

The conditional mean of r_t is given by:

$$E(r_t | r_s, s < t) = E(\sigma_t e_t | r_s, s < t) = \sigma_t E(e_t) = \sigma_t \cdot 0 = 0$$

Conditional variance of r_t is as follows:

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2$$

Hence $r_t | r_{t-1} \sim N(0, \alpha_0 + \alpha_1 r_{t-1}^2)$.

We can see from the above conditional variance equation, the current variance of r_t depends on the past value of r_{t-1} . It means when r_{t-1} has a large value at time $t - 1$, the variance at time t will be large too. It then implies that r_t will have large volatility at time t . This model set up is able to capture the volatility clustering phenomenon we observed in the Google stock return data.

3.3 r_t is White Noise

The unconditional mean of r_t is as follows:

$$E[E(r_t | r_s, s < t)] = 0$$

The unconditional variance of r_t can be derived by taking the expectation of the equation from Section 3.1.

$$\begin{aligned}
Var(r_t) &= E(r_t^2) = E(e_t^2(\alpha_0(1 + \sum_{j=1}^{\infty} \alpha_1^j e_{t-1}^2 \cdots e_{t-j}^2))) \\
&= \alpha_0(1 + \alpha_1^1 + \alpha_1^2 + \cdots + \alpha_1^{\infty}) \\
&= \frac{\alpha_0}{1 - \alpha_1}, \quad \text{where } 0 \leq \alpha_1 < 1
\end{aligned}$$

The autocovariance of r_t is:

$$\begin{aligned}
cov(r_{t+h}, r_t) &= E(r_{t+h}r_t) = E[E(r_{t+h}r_t | r_s, s < t+h)] \\
&= E[r_t E(r_{t+h} | r_s, s < t+h)] \\
&= E[r_t \cdot 0] \\
&= 0, \quad \text{for all } h > 0
\end{aligned}$$

Hence, the unconditional distribution of r_t can be written as follows:

$$r_t \sim \text{white noise} \left(0, \frac{\alpha_0}{1 - \alpha_1} \right)$$

We can see from the above result, r_t is unconditionally homoscedastic when $0 \leq \alpha_1 < 1$. So when r_t is stationary ($0 \leq \alpha_1 < 1$), r_t is conditionally heteroskedastic but unconditionally homoscedastic.

Although we are using asset returns for our ARCH model example, the original ARCH model was used for modeling squared error terms from a time series model. That's why the ARCH model has this nice unconditional property as white noise.

3.4 r_t Is Not i.i.d.

Although r_t has 0 covariance to its past values, r_t^2 does depend on the squared past returns. It can be shown as follows:

$$\begin{aligned}
E(r_t^2 | r_{t-1}) &= E(\alpha_0 e_t^2 + \alpha_1 e_t^2 r_{t-1}^2 | r_{t-1}) \\
&= \alpha_0 + \alpha_1 r_{t-1}^2
\end{aligned}$$

So we can see ARCH(1) is a case where the elements in the process have 0 covariance, but the elements are not independent. This conclusion also confirms our previous knowledge that two variables with 0 covariance do not mean they are independent.

3.5 r_t^2 IS A non-Gaussian AR(1) Process

From the ARCH(1) definition, we can take the square of r_t and rewrite the equations as follows:

$$\begin{aligned}
r_t^2 &= \sigma_t^2 e_t^2 \\
\alpha_0 + \alpha_1 r_{t-1}^2 &= \sigma_t^2
\end{aligned}$$

We then subtract the second equation from the first equation to get the following equation:

$$\begin{aligned} r_t^2 - (\alpha_0 + \alpha_1 r_{t-1}^2) &= \sigma_t^2(e_t^2 - 1) \\ \Rightarrow r_t^2 &= \alpha_0 + \alpha_1 r_{t-1}^2 + \nu_t \end{aligned}$$

where $\nu_t = \sigma_t^2(e_t^2 - 1)$ is non-normal white noise. $(e_t^2 - 1)$ follows a shifted Chi-Square distribution since e_t^2 is the square of a $N(0,1)$ random variable.

Also if $E(r_t^4) < \infty$, then r_t^2 has the same ACF as AR(1) process.

The ACF for r_t^2 is as follows:

$$\rho_{r^2}(h) = \alpha_1^{|h|}$$

for all h and $\alpha_1 < 1$.

With the knowledge that the squared asset return is also a stationary AR(1) process, we can use ACF and PACF plots of asset return and squared asset return to look for an ARCH(1) model. If the asset return time series plot exhibits a white noise pattern and the squared asset return PACF plot shows cut off pattern after lag 1, we can use an ARCH(1) model for modeling variance of the asset return. We will talk more about this in later sections.

3.6 r_t Has Heavier Tails Than Standard Normal Distribution If $\alpha_1 < \frac{1}{3}$

When $3\alpha_1 < 1$ and $E(r_t^4) < \infty$, we can show that kurtosis of r_t is greater than 3. It means that r_t has heavier tails than standard normal distribution. It is similar to our empirical observations that asset return usually has more outliers than a time series generated from normal distribution.

3.7 ARCH(m) Process

We have introduced ARCH(1) process in previous sections. We can extend the process to include return values from several pass time periods to form an ARCH(m) process. An ARCH(m) process is the process to predict current variance with squared asset return values going back to m time periods. Here is the formal definition for ARCH(m) process:

$$r_t = \sigma_t e_t$$

Where $e_t \sim iidN(0, 1)$ and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \dots + \alpha_m r_{t-m}^2$$

With some restrictions on ARCH(m) coefficients, the squared asset returns will be AR(m).

4. ARCH(1) Simulation

In this section, let's take a look at simulation data to get a visual understanding of an ARCH(1) process.

Figure 3: ARCH(1) Simulation $n=1000$, $\alpha_0 = 5$ and $\alpha_1 = 0.5$

```
In [5]: # ARCH(1) Simulation n=1000, alpha_0=5 and alpha_1=0.5

# simulated process parameters
rs = np.random.RandomState([12345, 77777])
dist = Normal(seed=rs)
vol = ARCH(p=1)
repro_mod = ConstantMean(None, volatility=vol, distribution=dist)
params = pd.Series({"mu": 0.0, "omega": 5, "alpha[1]": 0.5, "beta[1]": 0.0})

# model simulation
arch1_sim = repro_mod.simulate(params, nobs=1000)
plt.plot(arch1_sim.data)
plt.show()
```

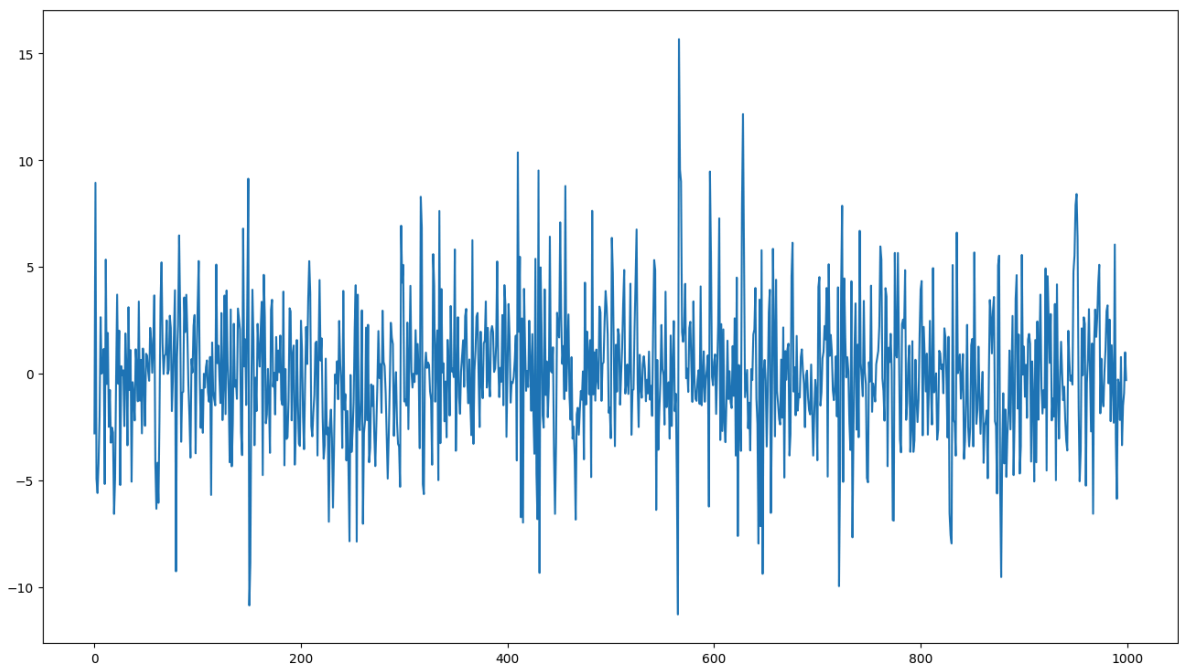


Figure 3 shows a simulated ARCH(1) process. From the graph, we can see the time series oscillates around 0 with occasional bursts of volatility. We can also see there are clusters of high volatility during the whole time period, like around $t = 600$. These features are all in agreement with the common features of financial asset returns we discussed earlier.

Figure 4: ACF and PACF of ARCH(1) Simulated Time Series

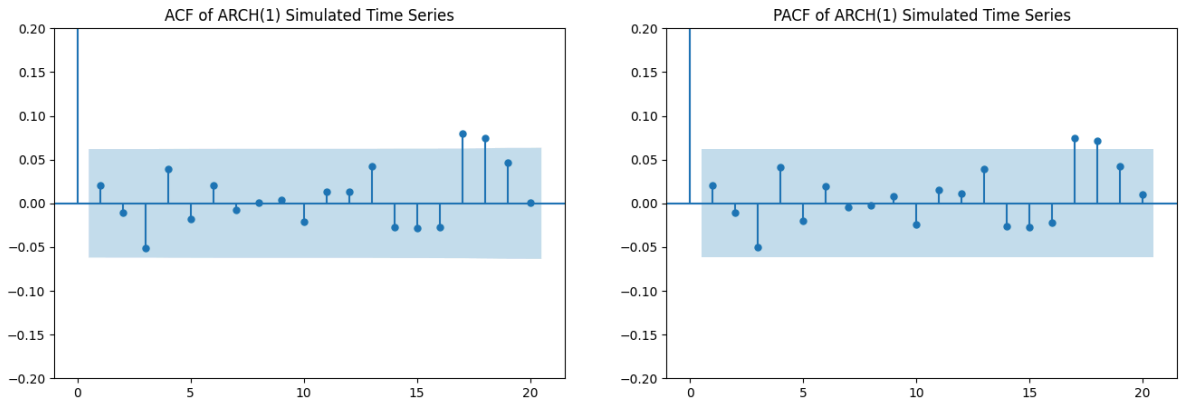
```
In [6]: # ACF and PACF of ARCH(1) Simulated Time Series
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
```

```

sm.graphics.tsa.plot_acf(arch1_sim.data.dropna(), lags=20, ax=ax1)
ax1.set(title="ACF of ARCH(1) Simulated Time Series")
ax1.set_ylim([-0.2, 0.2])

sm.graphics.tsa.plot_pacf(arch1_sim.data.dropna(), lags=20, ax=ax2)
ax2.set(title="PACF of ARCH(1) Simulated Time Series")
ax2.set_ylim([-0.2, 0.2])
plt.show()

```



The ACF and PACF plots in figure 4 shows that the simulated time series do not exhibit any correlations and behave quite like white noise.

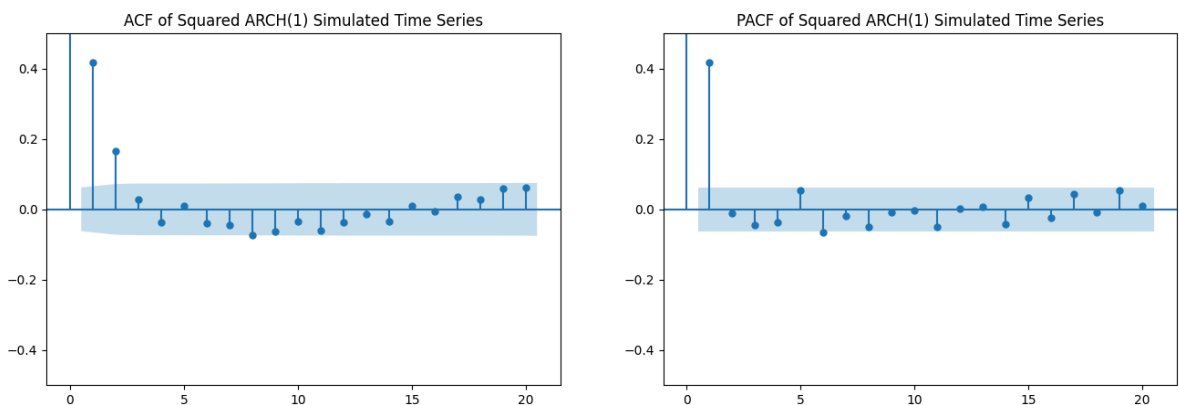
Figure 5: ACF and PACF of Squared ARCH(1) Simulated Time Series

```

In [7]: # ACF and PACF of Squared ARCH(1) Simulated Time Series
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(arch1_sim.data.dropna() ** 2, lags=20, ax=ax1)
ax1.set(title="ACF of Squared ARCH(1) Simulated Time Series")
ax1.set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_pacf(arch1_sim.data.dropna() ** 2, lags=20, ax=ax2)
ax2.set(title="PACF of Squared ARCH(1) Simulated Time Series")
ax2.set_ylim([-0.5, 0.5])
plt.show()

```



However, in figure 5, the PACF plot of squared simulated time series shows a drop after lag 1. The ACF shows a gradual decline. The two graphs suggest the properties of the AR(1) process for the squared simulated time series.

5. Conclusion

In this lesson, we went through the definition and properties of an ARCH(1) model and introduced a more general ARCH(m) process too. In the next lesson, we will introduce another general form of the ARCH process—the GARCH process. The GARCH process is widely used in finance research. We will go through the basics and demonstrate how to apply the GARCH model to a real finance problem.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Kempthorne, Peter et al. "Volatility Modeling." Topics in Mathematics with Applications in Finance, Fall 2013, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-s096-topics-in-mathematics-with-applications-in-finance-fall-2013/resources/mit18_s096f13_lecnote9/

- **Required pages:** Entire lecture
- **Estimated reading time:** 1 hour

FINANCIAL ECONOMETRICS

MODULE 5 | LESSON 2

GARCH MODEL

Reading Time 60 minutes

Prior Knowledge Basics of Time Series, ARIMA model, ARCH model

Keywords GARCH(1,1), GARCH(p,q), Engle's ARCH Lagrange Multiplier (LM) Test, Nyblom Stability Test, Sign Bias Test, Adjusted Pearson Goodness-of-Fit Test

In the last lesson, we discussed using the ARCH process to model non-constant variances in financial asset returns. We talked about the common features of financial asset returns, especially the existence of time-varying volatility in the data. We introduced the ARCH process to model non-constant variance specifically. We went through the key properties of the ARCH process. In this lesson, we are going to continue our journey of modeling non-constant variance. We are going to expand ARCH(1) model and introduce a general model: GARCH. We will go through the basics of GARCH and apply GARCH to Google stock returns.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from arch import arch_model
from scipy import stats
from statsmodels.stats.diagnostic import acorr_ljungbox

plt.rcParams["figure.figsize"] = (16, 9) # Figure size and width
```

```
In [2]: # Download the dataset
m5_data = pd.read_csv("M5. goog_eur_10.csv")

# Convert date variable to date format
m5_data["Date2"] = pd.to_datetime(m5_data["Date"], format="%m/%d/%Y")
goog = m5_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")

goog["GOOGLE_R"] = np.log(goog["GOOGLE"]).diff().dropna()
```

1. GARCH Model

1.1 GARCH(1,1) Model

GARCH stands for generalized autoregressive conditional heteroskedasticity, and it is an extension of the ARCH model. In the variance function of the ARCH model, a lagged variance term is added to the right of the equation to become a GARCH model. We can formally define GARCH(1,1) as follows:

$$r_t = \sigma_t e_t$$

Where $e_t \sim iidN(0, 1)$ and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

with $\alpha_0 > 0$, $0 \leq \alpha_1 < 1$ and $0 \leq \beta_1 < 1$

The requirements of $\alpha_0 > 0$, $0 \leq \alpha_1 < 1$ and $0 \leq \beta_1 < 1$ will make sure r_t is stationary and the variance is non-negative and finite.

Structurally, the relationship between GARCH and ARCH models is similar to that between ARMA and AR models. For ARCH(1) model, we can write it as follows:

$$r_t = e_t \sqrt{(\alpha_0 + \alpha_1 r_{t-1}^2)}$$

In the above ARCH(1) model, we see we use r_{t-1} to predict r_t . This concept is the same as for AR(1) model, so we sometimes call this term the AR part of the ARCH model.

Similarly, let's write a GARCH(1,1) model in the following form:

$$r_t = e_t \sqrt{(\alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2)}$$

In the GARCH(1,1) formula, we not only have r_{t-1} to predict r_t but also have σ_{t-1} to predict r_t . The reason why GARCH(1,1) is like ARMA(1,1) is because we have r_{t-1} like AR part and σ_{t-1} like MA part for GARCH.

So why do we need the GARCH model? Even though the ARCH model seems to be able to cover all the features of asset returns, it usually requires many parameters to properly model the asset return. In order to tackle this issue, the GARCH model was developed so that there is no need to include too many terms in a model.

1.2 Properties of GARCH(1,1) Model

Most of the properties for the GARCH(1,1) model are similar to the ARCH(1,1) model. In this section, we will highlight the key properties for GARCH(1,1).

1.2.1 r_t is Conditionally Normally Distributed

The above statement follows the same proof as we describe in ARCH(1) model. We can summarize it in the following formula:

$$r_t|r_{t-1} \sim N(0, \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2)$$

1.2.2 r_t^2 is An ARMA(1,1) Process

We know that GARCH(1,1) is defined as follows:

$$r_t = \sigma_t e_t \quad (1)$$

and

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2 \quad (2)$$

If we subtract the equation (1) and equation (2), we can get the following new equation:

$$r_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2 + \nu_t$$

where $\nu_t = \sigma_t^2(e_t^2 - 1)$ and ν_t is white noise.

Since $\nu_{t-1} = \sigma_{t-1}^2(e_{t-1}^2 - 1)$ and $r_{t-1}^2 = \sigma_{t-1}^2 e_{t-1}^2$, we can get $\sigma_{t-1}^2 = r_{t-1}^2 - \nu_{t-1}$. Plug this last result into the new equation from the subtraction of two equations from the definition, we can get the following equation:

$$r_t^2 = \alpha_0 + (\alpha_1 + \beta_1) r_{t-1}^2 + \nu_t - \beta_1 \nu_{t-1}$$

The above equation is an ARMA(1,1) with the squared asset return as the AR part and white noise as the MA part. To ensure stationarity, $0 < \alpha_1 + \beta_1 < 1$.

1.2.3 r_t is White Noise

Using the same calculation from the ARCH model, we can show that GARCH(1,1) is white noise. We can write unconditional r_t distribution as follows:

$$r_t \sim \text{white noise} \left(0, \frac{\alpha_0}{1 - (\alpha_1 + \beta_1)} \right)$$

where $0 < \alpha_1 + \beta_1 < 1$

This shows similar results as the ARCH(1) model. r_t is unconditionally homoscedastic when $\alpha_1 + \beta_1 < 1$. So when r_t is stationary ($0 \leq \alpha_1 < 1$), r_t is conditionally heteroskedastic but unconditionally homoscedastic. Also, even though we use asset return as an example for GARCH(1,1) here, it was originally applied to the squared error term from a time series regression model.

1.2.4 GARCH(1,1) Has Infinite Memory

By iterating the variance equation from GARCH(1,1), we can write the equation as follows:

$$\begin{aligned}
\sigma_{t^2} &= \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2 \\
&= \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 (\alpha_0 + \alpha_1 r_{t-2}^2 + \beta_1 \sigma_{t-2}^2) \\
&= \alpha_0 + \alpha_0 \beta_1 + \alpha_1 r_{t-1}^2 + \alpha_1 \beta_1 r_{t-2}^2 + \beta_1^2 (\alpha_0 + \alpha_1 r_{t-3}^2 + \beta_1 \sigma_{t-3}^2) \\
&= \alpha_0 (1 + \beta_1 + \beta_1^2 + \beta_1^3 + \dots) + \alpha_1 \sum_{i=1}^{\infty} \beta_1^{i-1} r_{t-i}^2
\end{aligned}$$

We can see that σ_{t^2} can be expanded as the infinite sum of past asset returns. This means that all the past asset returns still have influence on today's conditional variance.

Alternatively, we can say GARCH(1,1) has infinite memory. All the coefficients should decay in order to maintain stationarity of the GARCH(1,1) process. Because of this feature, the GARCH model is usually better to model time series data than the ARCH(1) model.

1.3 GARCH(p, q) Model

As with other time series models we have talked about so far, we can easily extend GARCH(1,1) model to GARCH(p, q) model.

We can define GARCH(p, q) as follows:

$$r_t = \sigma_t e_t$$

where $e_t \sim iidN(0, 1)$ and the variance equation is:

$$\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \dots + \alpha_p r_{t-p}^2 + \beta_1 \sigma_{t-1}^2 + \beta_2 \sigma_{t-2}^2 + \dots + \beta_q \sigma_{t-q}^2$$

where $\alpha_0 > 0$, $\alpha_i \geq 0$ for $i = 1, \dots, p$ and $\beta_j \geq 0$ for $j = 1, \dots, q$

We can now rewrite the variance equation with a backshift operator as follows:

$$\sigma_t^2 = \alpha_0 + \alpha(B) r_t^2 + \beta(B) \sigma_t^2$$

Where

$$\alpha(B) = \alpha_1 B + \alpha_2 B^2 + \dots + \alpha_p B^p$$

and

$$\beta(B) = \beta_1 B + \beta_2 B^2 + \dots + \beta_q B^q$$

For the above characteristic equations, the absolute values of the roots of B must be greater than 1 to ensure that r_t is stationary.

2. Model Diagnostics

There are a number of tests that are usually used to check for ARCH and GARCH models. We will cover some of them in this section.

2.1 Ljung-Box Test

We introduced this test in last module. The Ljung-Box test is used to see if there is serial correlation in a time series. We use this test if there is an overall correlation in the residuals or squared residuals from a time series model, including ARCH and GARCH models. The null hypothesis (H_0) is there is no serial correlation in the time series. Hence, if the p -value < 0.05 , we will reject the null hypothesis, and the time series has a serial correlation issue.

2.2 Information Criteria

In the last module, we also introduced Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC). These are the metrics used to evaluate overall model specifications. The lower the numbers of AIC and/or BIC, the better the model specifications. There are other information criteria, but we usually focus on AIC or BIC.

2.3 Log Likelihood Number

We also use the log likelihood number to evaluate the goodness-of-fit of a model if it is fitted with the maximum likelihood method. The lower the log likelihood number, the better the model fit.

2.4 Engle's ARCH Lagrange Multiplier (LM) Test

Engle's ARCH LM test, or Engle's ARCH test, determines if there is a serial correlation (autocorrelation) issue in the squared residuals of a time series model. The null hypothesis (H_0) is that there is no serial correlation in the squared residuals.

2.5 Nyblom Stability Test

This is the test to check if there is a regime change for the time series over time. The test checks if there is any structural change among variables or if coefficients are stable with no variation overtime. The null hypothesis (H_0) is that there is no regime change and the variances for coefficients are 0.

2.6 Sign Bias Test

This is the test to check if shocks not modeled by the model have an impact on volatilities. The sign bias test is used to determine the impact of large and small shocks on volatility. The negative sign bias test is to check if negative shocks not modeled by the model have an

impact on volatility. The positive sign bias test is to check if positive shocks not modeled by the model have an impact on volatility. The null hypothesis (H_0) is that there is no impact from shocks not modeled by the model.

2.7 Adjusted Pearson Goodness-of-Fit Test

This test compares the empirical distribution of standard residuals with the selected theoretical distribution. It is a chi-squared test. The null hypothesis (H_0) is that the empirical distribution of the standard residuals is equal to theoretical distribution.

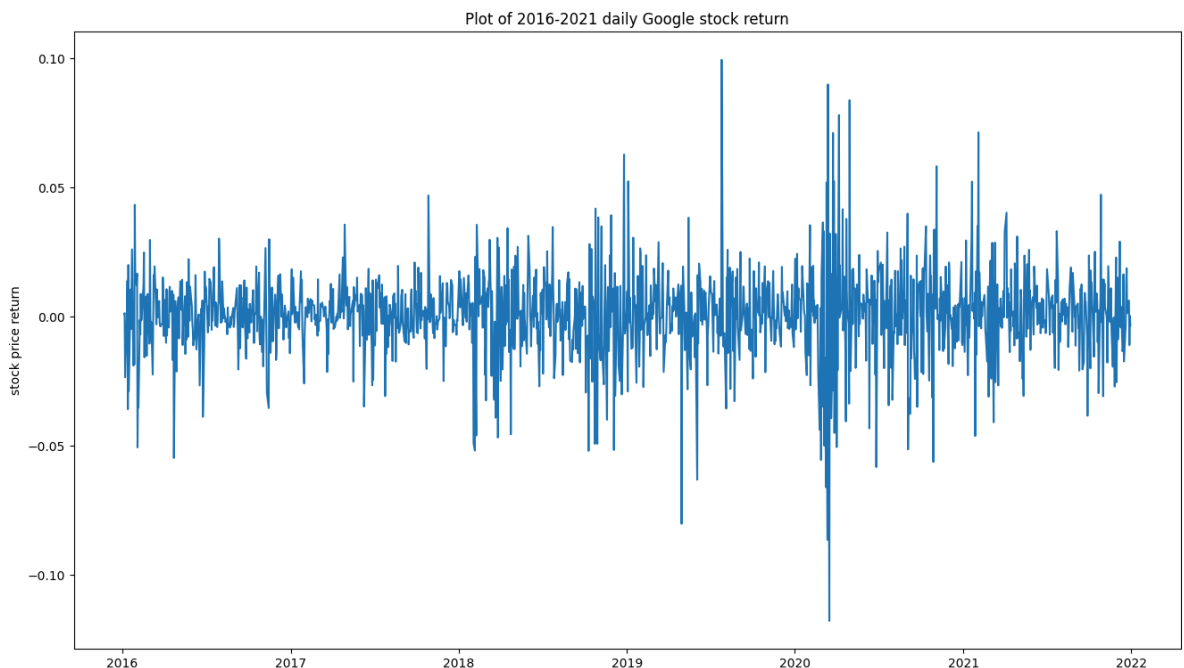
3. GARCH Application: Google Stock Return

In this section, we are going to use Google stock return data from 2016 to 2021 to run a GARCH model. We'll start by looking into the data first before building the model.

3.1 Data Exploration

Figure 1: Google Stock Returns from 2016 to 2021

```
In [3]: # Google Stock Returns from 2016 to 2021
plt.plot(goog.GOOGLE_R)
plt.ylabel("stock price return")
plt.title("Plot of 2016-2021 daily Google stock return")
plt.show()
```



In figure 1, we can see Google stock returns are oscillating around 0. We can see there are periods of time when the volatilities of returns are higher than other times. This pattern of

volatility in Google stock return data is the phenomenon that the GARCH model tries to capture.

Figure 2: Histogram of Google Stock Returns

```
In [4]: # Histogram of Google Stock Returns
plt.figure(figsize=(11, 5))
goog_r = goog.GOOGLE_R.dropna()

x = np.linspace(min(goog_r), max(goog_r), len(goog_r))
values, bins, _ = plt.hist(goog_r, bins=25) # Histogram

(mu, sigma) = stats.norm.fit(goog_r)
plt.plot(x, stats.norm.pdf(x, mu, sigma) * sum(values * np.diff(bins)), "r") # Den

plt.title("Google Stock Return Histogram")
plt.show()
```

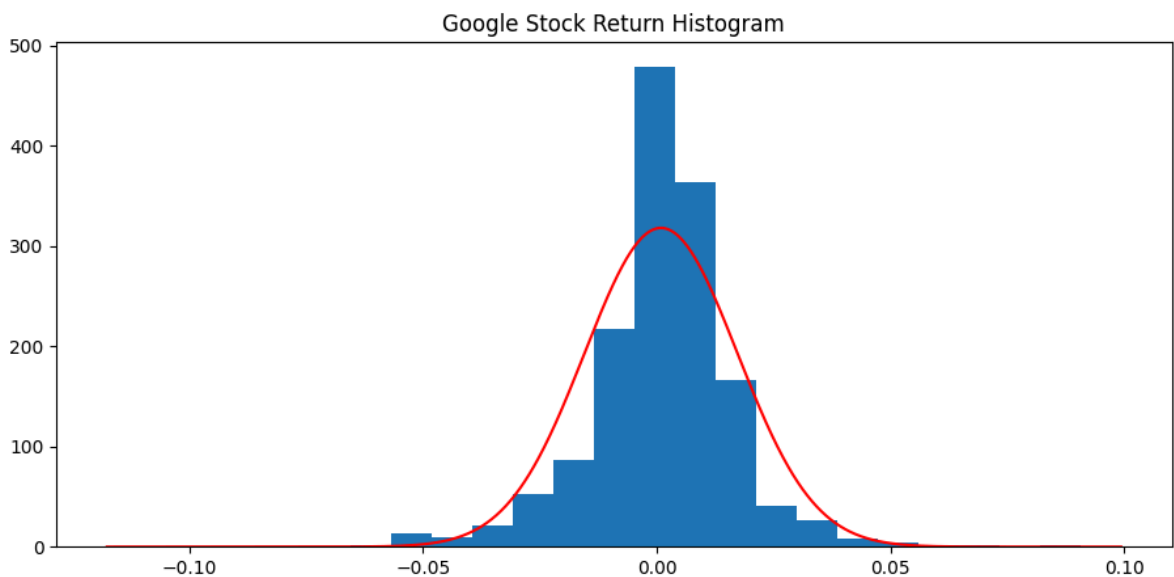
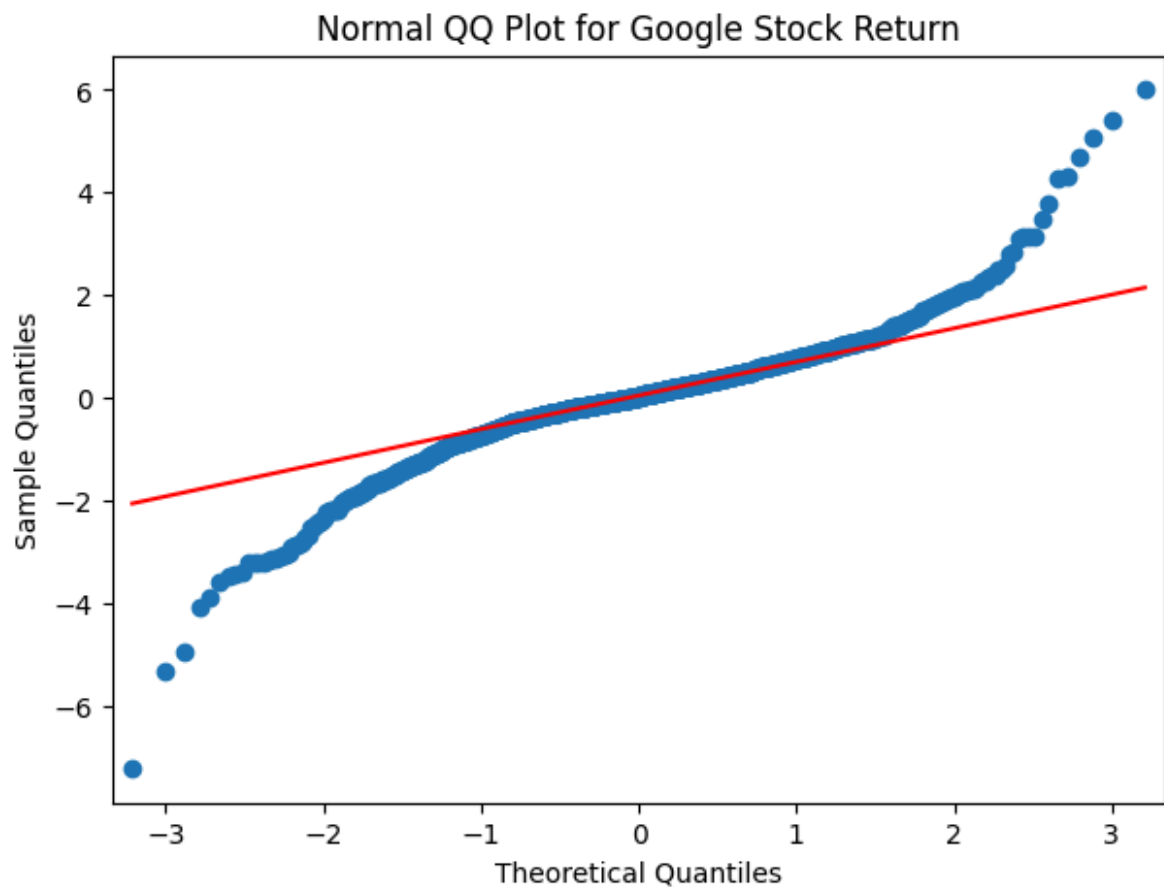


Figure 2 shows the histogram of Google's stock returns and a normal distribution curve overlaid on it. We can see Google's stock return distribution has a higher peak and fatter tail than normal distribution.

Figure 3: Normal QQ Plot for Google Stock Returns

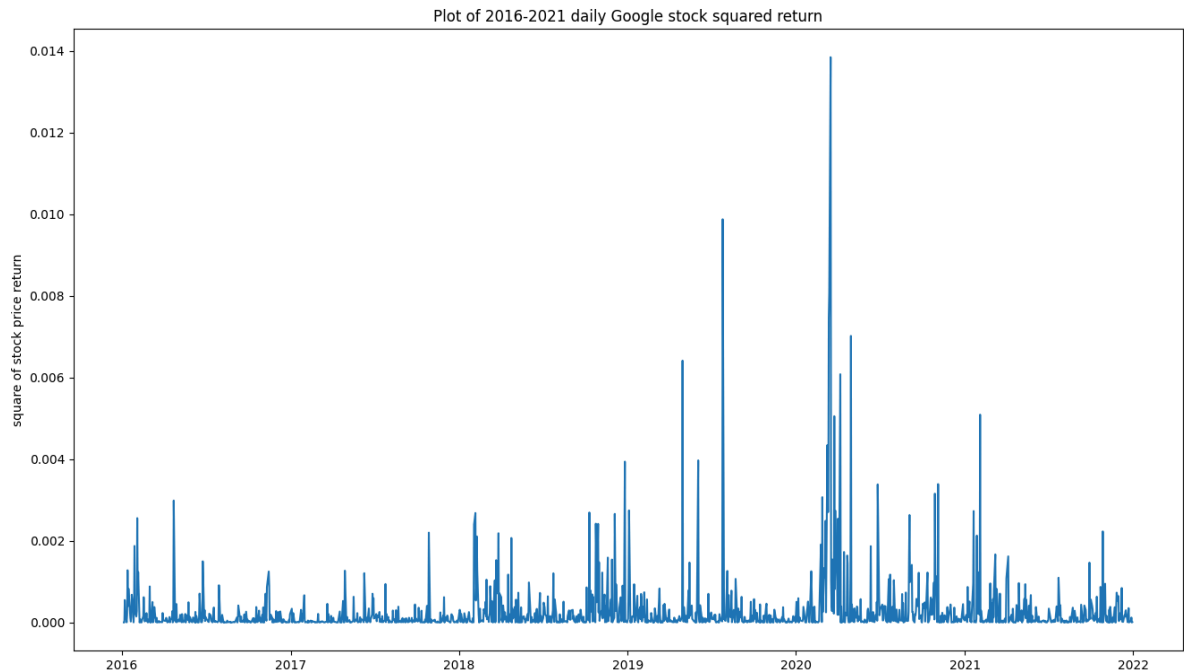
```
In [5]: # Normal QQ Plot for Google Stock Returns
qq = sm.qqplot(goog_r, stats.norm, fit=True, line="q")
qq.set_size_inches((7, 5))
plt.title("Normal QQ Plot for Google Stock Return")
plt.show()
```



In figure 3, we can clearly see that Google stock returns have a fat-tailed distribution compared to normal distribution.

Figure 4: Google Stock Squared Returns from 2016 to 2021

```
In [6]: # Google Stock Squared Returns
plt.plot(goog.GOOGLE_R**2)
plt.ylabel("square of stock price return")
plt.title("Plot of 2016-2021 daily Google stock squared return")
plt.show()
```



From figure 4, we can see there are serial correlations to the squared return for Google stock. The high returns are usually followed by more volatilities.

Figure 5: ACF and PACF Plots for Google Stock Returns and Squared Returns

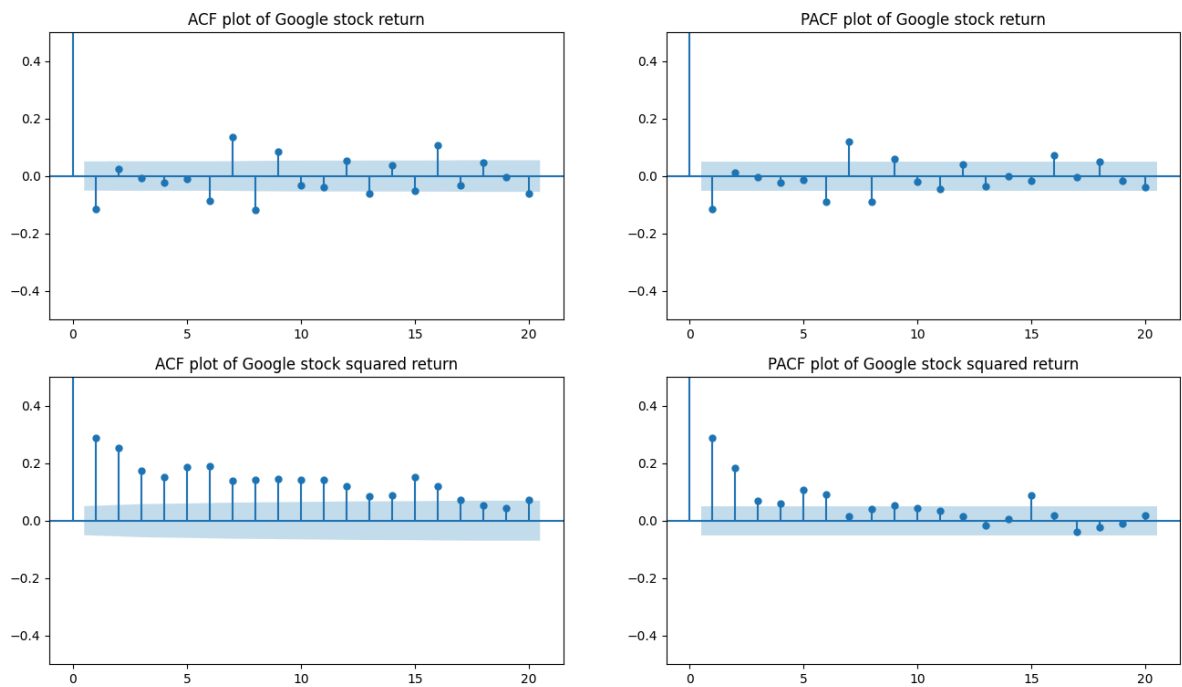
```
In [7]: # ACF and PACF Plots for Google Stock Returns and Squared Returns
fig, ax = plt.subplots(2, 2)

sm.graphics.tsa.plot_acf(goog.GOOGLE_R.dropna(), lags=20, ax=ax[0, 0])
ax[0, 0].set(title="ACF plot of Google stock return")
ax[0, 0].set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_pacf(goog.GOOGLE_R.dropna(), lags=20, ax=ax[0, 1])
ax[0, 1].set(title="PACF plot of Google stock return")
ax[0, 1].set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_acf(goog.GOOGLE_R.dropna() ** 2, lags=20, ax=ax[1, 0])
ax[1, 0].set(title="ACF plot of Google stock squared return")
ax[1, 0].set_ylim([-0.5, 0.5])

sm.graphics.tsa.plot_pacf(goog.GOOGLE_R.dropna() ** 2, lags=20, ax=ax[1, 1])
ax[1, 1].set(title="PACF plot of Google stock squared return")
ax[1, 1].set_ylim([-0.5, 0.5])
plt.show()
```



In figure 5, ACF and PACF plots for Google stock returns and squared returns all show patterns like the ARMA model. Hence, we can start with the GARCH(1,1) model for model specifications.

3.2 Model Estimation and Diagnostics

Let's start with the GARCH(1,1) model using Google stock return data. Figure 6 shows the GARCH(1,1) model result.

Figure 6: GARCH(1,1) Model with Normal White Noise

```
In [8]: # GARCH(1,1) Model with Normal White Noise
garch11_spec = arch_model(
    goog.GOOGLE_R.dropna(),
    vol="GARCH",
    p=1,
    q=1,
    mean="AR",
    dist="Normal",
    rescale=True,
)
garch11_fit = garch11_spec.fit()
garch11_fit.summary()
```

```

Iteration:      1,   Func. Count:      6,   Neg. LLF: 18102370089.838078
Iteration:      2,   Func. Count:     14,   Neg. LLF: 199289710361.31866
Iteration:      3,   Func. Count:     23,   Neg. LLF: 2905.730302628701
Iteration:      4,   Func. Count:     31,   Neg. LLF: 2886.925378396835
Iteration:      5,   Func. Count:     38,   Neg. LLF: 2728.1622986296334
Iteration:      6,   Func. Count:     43,   Neg. LLF: 2727.9777720000106
Iteration:      7,   Func. Count:     48,   Neg. LLF: 2727.9469065715693
Iteration:      8,   Func. Count:     53,   Neg. LLF: 2727.9393448273977
Iteration:      9,   Func. Count:     58,   Neg. LLF: 2727.937888260677
Iteration:     10,   Func. Count:     63,   Neg. LLF: 2727.9370769987204
Iteration:     11,   Func. Count:     68,   Neg. LLF: 2727.937067484191
Iteration:     12,   Func. Count:     72,   Neg. LLF: 2727.9370674840043

```

Optimization terminated successfully (Exit mode 0)

Current function value: 2727.937067484191

Iterations: 12

Function evaluations: 72

Gradient evaluations: 12

Out[8]:

AR - GARCH Model Results

Dep. Variable:	GOOGLE_R	R-squared:	0.000
Mean Model:	AR	Adj. R-squared:	0.000
Vol Model:	GARCH	Log-Likelihood:	-2727.94
Distribution:	Normal	AIC:	5463.87
Method:	Maximum Likelihood	BIC:	5485.14
No. Observations:			1504
Date:	Tue, Dec 09 2025	Df Residuals:	1503
Time:	07:14:08	Df Model:	1

Mean Model

	coef	std err	t	P> t 	95.0% Conf. Int.
Const	0.1114	4.019e-02	2.772	5.578e-03	[3.262e-02, 0.190]

Volatility Model

	coef	std err	t	P> t 	95.0% Conf. Int.
omega	0.1250	9.366e-02	1.335	0.182	[-5.858e-02, 0.309]
alpha[1]	0.0822	3.786e-02	2.170	2.999e-02	[7.959e-03, 0.156]
beta[1]	0.8671	6.959e-02	12.461	1.223e-35	[0.731, 1.004]

Covariance estimator: robust

```

In [9]: # Diagnostic tests for GARCH(1,1) Model with Normal White Noise
print("GARCH(1,1) Model with Normal White Noise\n")

```

```
# Ljung-Box test and the Box-Pierce test
print("Ljung-Box and Box-Pierce tests on stanrdized residuals")
print(acorr_ljungbox(garch11_fit.std_resid, boxpierce=True))

print("\nLjung-Box and Box-Pierce tests on stanrdized squared residuals")
print(acorr_ljungbox(garch11_fit.std_resid**2, boxpierce=True))

# ARCH LM test for conditional heteroskedasticity
print("\nARCH LM test for conditional heteroskedasticity")
print(garch11_fit.arch_lm_test(standardized=True))
```

GARCH(1,1) Model with Normal White Noise

Ljung-Box and Box-Pierce tests on stanrdized residuals

	lb_stat	lb_pvalue	bp_stat	bp_pvalue
1	0.453025	0.500902	0.452122	0.501329
2	0.576445	0.749595	0.575215	0.750056
3	1.467876	0.689704	1.463686	0.690677
4	2.000287	0.735706	1.993976	0.736867
5	6.544690	0.256763	6.517257	0.259087
6	7.905635	0.245099	7.870972	0.247710
7	13.250117	0.066244	13.183515	0.067761
8	17.537745	0.024972	17.442673	0.025816
9	17.725621	0.038494	17.629176	0.039728
10	18.162461	0.052284	18.062536	0.053918

Ljung-Box and Box-Pierce tests on stanrdized squared residuals

	lb_stat	lb_pvalue	bp_stat	bp_pvalue
1	0.006414	0.936170	0.006401	0.936234
2	0.101005	0.950752	0.100741	0.950877
3	0.103420	0.991424	0.103148	0.991457
4	0.115280	0.998401	0.114961	0.998410
5	0.636968	0.986248	0.634224	0.986383
6	0.641445	0.995669	0.638677	0.995720
7	1.616170	0.978013	1.607577	0.978350
8	2.954319	0.937191	2.936840	0.938258
9	3.117530	0.959452	3.098859	0.960246
10	3.119505	0.978470	3.100818	0.978951

ARCH LM test for conditional heteroskedasticity

ARCH-LM Test

H0: Standardized residuals are homoskedastic.

ARCH-LM Test

H1: Standardized residuals are conditionally heteroskedastic.

Statistic: 15.7513

P-value: 0.8968

Distributed: <scipy.stats._distn_infrastructure.rv_continuous_frozen object at 0x7ac b2363b550>

Figure 6 shows the GARCH(1,1) model with normal white noise. Everything looks good from the model results except that we can see one of the coefficient estimates is not significant. Now let's look on some diagnostic plots.

Figure 7: Model Diagnostic Plots for the GARCH(1,1) Model with Normal White Noise

```

In [10]: # Model Diagnostic Plots for the GARCH(1,1) Model with Normal White Noise
fig, ax = plt.subplots(4, 3, figsize=(16, 12))

# Figure Row 1 Column 1
ax[0, 0].plot(goog.GOOGLE_R)
ax[0, 0].plot(2.0 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].plot(-2.0 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].tick_params(labelrotation=45)
ax[0, 0].set_title("Series with 2 conditional SD")
ax[0, 0].set_ylabel("Return")

# Figure Row 1 Column 2
VaR_1 = stats.norm.ppf(0.99)
ax[0, 1].plot(goog.GOOGLE_R)
ax[0, 1].plot(VaR_1 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].plot(-VaR_1 * garch11_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].tick_params(labelrotation=45)
ax[0, 1].set_title("Series with 1% VaR Limits")
ax[0, 1].set_ylabel("Return")

# Figure Row 1 Column 3
ax[0, 2].plot(garch11_fit.conditional_volatility / 100.0)
ax[0, 2].set_title("Conditional SD")
ax[0, 2].tick_params(labelrotation=45)

# Figure Row 2 Column 1
sm.graphics.tsa.plot_acf(garch11_fit.resid / 100.0, lags=20, ax=ax[1, 0])
ax[1, 0].set_title("ACF of Observations")
ax[1, 0].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 2
sm.graphics.tsa.plot_acf(garch11_fit.resid**2, lags=20, ax=ax[1, 1])
ax[1, 1].set_title("ACF of Squared Observations")
ax[1, 1].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 3
sm.graphics.tsa.plot_acf(np.abs(garch11_fit.resid), lags=20, ax=ax[1, 2])
ax[1, 2].set_title("ACF of Absolute Observations")
ax[1, 2].set_ylim([-0.3, 0.3])

# Figure Row 3 Column 1
ax[2, 0].xcorr(
    garch11_fit.resid**2,
    garch11_fit.resid,
    usevlines=True,
    maxlags=30,
    normed=True,
    lw=2,
)
ax[2, 0].set_title("Cross-Correlation of Squared Observations \n vs Actual Observat

# Figure Row 3 Column 2
standaraized_residuals = garch11_fit.std_resid
min_val = np.min(standaraized_residuals)
max_val = np.max(standaraized_residuals)

```

```

empirical_density = np.linspace(min_val, max_val, len(standaraized_residuals))
ax[2, 1].plot(empirical_density, stats.norm.pdf(empirical_density), lw=1)
ax[2, 1].set_title("Empirical density of \n standardized residuals")

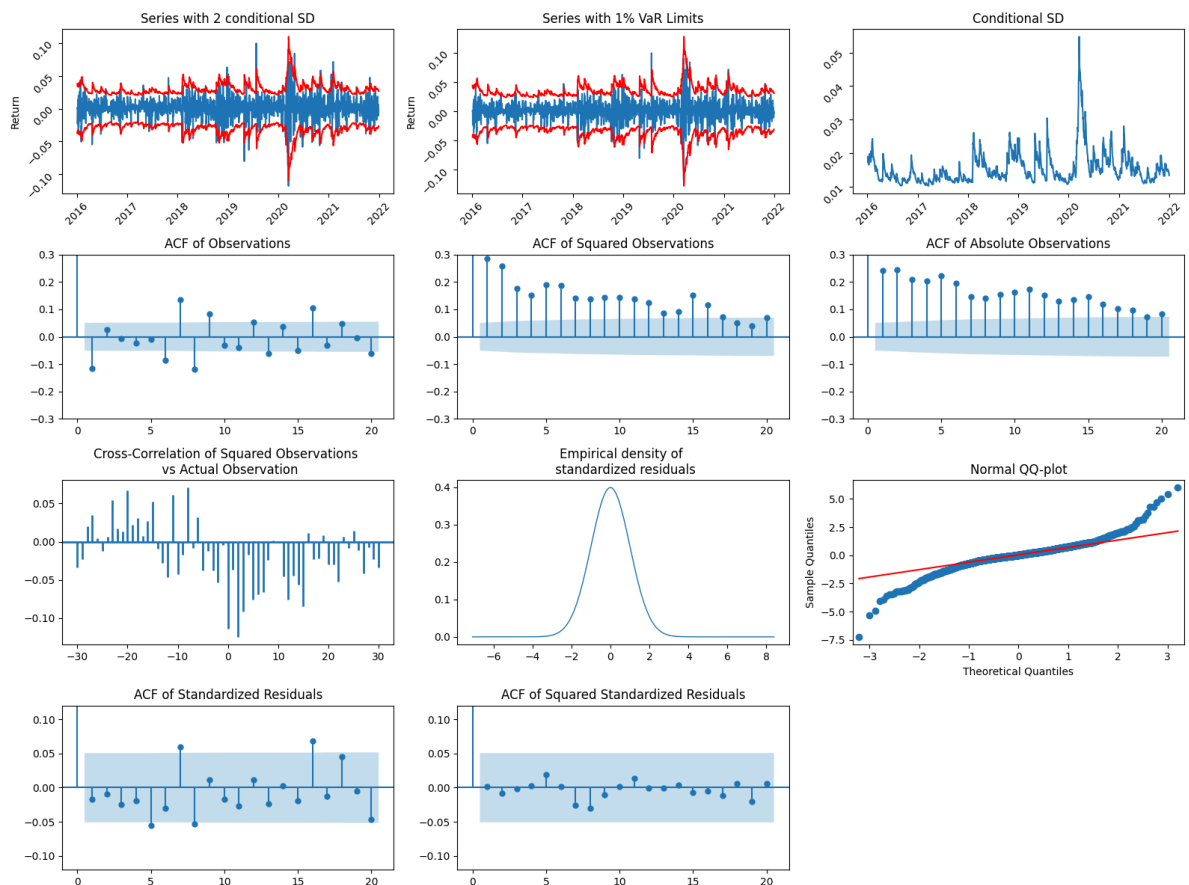
# Figure Row 3 Column 3
sm.qqplot(garch11_fit.resid, stats.norm, fit=True, line="q", ax=ax[2, 2])
ax[2, 2].set_title("Normal QQ-plot")

# Figure Row 4 Column 1
sm.graphics.tsa.plot_acf(garch11_fit.std_resid, lags=20, ax=ax[3, 0])
ax[3, 0].set_title("ACF of Standardized Residuals")
ax[3, 0].set_ylim([-0.12, 0.12])

# Figure Row 4 Column 2
sm.graphics.tsa.plot_acf((garch11_fit.std_resid) ** 2, lags=20, ax=ax[3, 1])
ax[3, 1].set_title("ACF of Squared Standardized Residuals")
ax[3, 1].set_ylim([-0.12, 0.12])

ax[3, 2].axis("off")
fig.tight_layout()
plt.show()

```



From figure 7, we can see that normal QQ plot for residuals is still showing fat-tailed distribution. With this piece of information, let's try a GARCH(1,1) model with Student's t -distribution to see if we can improve the model.

Figure 8: GARCH(1,1) Model with Student's t White Noise

In [11]: *# GARCH(1,1) Model with Student's t White Noise*

```
garch11_t_spec = arch_model(  
    goog.GOOGLER.dropna(),  
    vol="GARCH",  
    p=1,  
    q=1,  
    mean="AR",  
    dist="StudentsT", # power=2.0,  
    rescale=True,  
)  
garch11_t_fit = garch11_t_spec.fit()  
garch11_t_fit.summary()
```

Iteration:	1,	Func. Count:	7,	Neg. LLF:	44460.09922944116
Iteration:	2,	Func. Count:	16,	Neg. LLF:	27005.829998976977
Iteration:	3,	Func. Count:	24,	Neg. LLF:	4643.085422573049
Iteration:	4,	Func. Count:	32,	Neg. LLF:	3989.0180594529356
Iteration:	5,	Func. Count:	41,	Neg. LLF:	3675.46031225776
Iteration:	6,	Func. Count:	48,	Neg. LLF:	2649.5004902070314
Iteration:	7,	Func. Count:	55,	Neg. LLF:	2766.3934879003095
Iteration:	8,	Func. Count:	62,	Neg. LLF:	2605.904525379311
Iteration:	9,	Func. Count:	68,	Neg. LLF:	2605.613036755023
Iteration:	10,	Func. Count:	75,	Neg. LLF:	2605.0821220917583
Iteration:	11,	Func. Count:	81,	Neg. LLF:	2604.9510046012065
Iteration:	12,	Func. Count:	87,	Neg. LLF:	2604.886371047379
Iteration:	13,	Func. Count:	93,	Neg. LLF:	2604.8534889473412
Iteration:	14,	Func. Count:	99,	Neg. LLF:	2604.8450921073613
Iteration:	15,	Func. Count:	105,	Neg. LLF:	2604.8447865363432
Iteration:	16,	Func. Count:	111,	Neg. LLF:	2604.844781341683
Iteration:	17,	Func. Count:	116,	Neg. LLF:	2604.844781341674

Optimization terminated successfully (Exit mode 0)

Current function value: 2604.844781341683

Iterations: 17

Function evaluations: 116

Gradient evaluations: 17

Out[11]:

AR - GARCH Model Results

Dep. Variable:	GOOGLE_R	R-squared:	0.000
Mean Model:	AR	Adj. R-squared:	0.000
Vol Model:	GARCH	Log-Likelihood:	-2604.84
Distribution:	Standardized Student's t	AIC:	5219.69
Method:	Maximum Likelihood	BIC:	5246.27
No. Observations:			1504
Date:	Tue, Dec 09 2025	Df Residuals:	1503
Time:	07:14:10	Df Model:	1

Mean Model

	coef	std err	t	P> t	95.0% Conf. Int.
Const	0.1487	2.799e-02	5.313	1.081e-07	[9.383e-02, 0.204]

Volatility Model

	coef	std err	t	P> t	95.0% Conf. Int.
omega	0.0616	2.976e-02	2.071	3.838e-02	[3.297e-03, 0.120]
alpha[1]	0.1039	2.528e-02	4.111	3.943e-05	[5.438e-02, 0.153]
beta[1]	0.8865	2.642e-02	33.552	8.423e-247	[0.835, 0.938]

Distribution

	coef	std err	t	P> t	95.0% Conf. Int.
nu	3.6441	0.364	10.013	1.337e-23	[2.931, 4.357]

Covariance estimator: robust

```
In [12]: # Diagnostic tests for GARCH(1,1) Model with Student's t White Noise
print("GARCH(1,1) Model with StudentsT White Noise\n")

# Ljung-Box test and the Box-Pierce test
print("Ljung-Box and Box-Pierce tests on standardized residuals")
print(acorr_ljungbox(garch11_t_fit.std_resid, boxpierce=True))

print("\nLjung-Box and Box-Pierce tests on standardized squared residuals")
print(acorr_ljungbox(garch11_t_fit.std_resid**2, boxpierce=True))

# ARCH LM test for conditional heteroskedasticity
print("\nARCH LM test for conditional heteroskedasticity for")
print(garch11_t_fit.arch_lm_test(standardized=True))
```

GARCH(1,1) Model with Student's t White Noise

Ljung-Box and Box-Pierce tests on standardized residuals

	lb_stat	lb_pvalue	bp_stat	bp_pvalue
1	0.053823	0.816539	0.053716	0.816719
2	0.204105	0.902982	0.203598	0.903211
3	1.270875	0.736061	1.266827	0.737025
4	1.718776	0.787304	1.712944	0.788365
5	6.361299	0.272633	6.333888	0.275072
6	7.381354	0.287014	7.348524	0.289815
7	11.470263	0.119381	11.412997	0.121591
8	15.396911	0.051872	15.313572	0.053327
9	15.404727	0.080402	15.321331	0.082479
10	15.902260	0.102465	15.814899	0.105055

Ljung-Box and Box-Pierce tests on standardized squared residuals

	lb_stat	lb_pvalue	bp_stat	bp_pvalue
1	0.010544	0.918215	0.010523	0.918296
2	0.327440	0.848980	0.326578	0.849346
3	0.403562	0.939506	0.402447	0.939737
4	0.404217	0.982130	0.403099	0.982222
5	0.530913	0.990948	0.529206	0.991015
6	0.620153	0.996055	0.617972	0.996093
7	1.733271	0.973120	1.724438	0.973509
8	3.345261	0.910862	3.325724	0.912287
9	3.586340	0.936473	3.565042	0.937644
10	3.740119	0.958310	3.717596	0.959189

ARCH LM test for conditional heteroskedasticity for

ARCH-LM Test

H0: Standardized residuals are homoskedastic.

ARCH-LM Test

H1: Standardized residuals are conditionally heteroskedastic.

Statistic: 13.3881

P-value: 0.9593

Distributed: <scipy.stats._distn_infrastructure.rv_continuous_frozen object at 0x7ac b231d7050>

In figure 8, now all the coefficient estimates are significant. Let's check the model diagnostic plots.

Figure 9: GARCH(1,1) Model with Student's t White Noise

```
In [13]: # Model Diagnostic Plots for the GARCH(1,1) Model with Student's t White Noise
fig, ax = plt.subplots(4, 3, figsize=(16, 12))

# Figure Row 1 Column 1
ax[0, 0].plot(goog.GOOGLE_R)
ax[0, 0].plot(2.0 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].plot(-2.0 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 0].tick_params(labelrotation=45)
ax[0, 0].set_title("Series with 2 conditional SD")
ax[0, 0].set_ylabel("Return")

# Figure Row 1 Column 2
```

```

VaR_1 = stats.t(df=len(goog.GOOGLE_R) - 1).ppf(0.99)
# VaR_1 = stats.norm.ppf(0.99)
ax[0, 1].plot(goog.GOOGLE_R)
ax[0, 1].plot(VaR_1 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].plot(-VaR_1 * garch11_t_fit.conditional_volatility / 100.0, c="r")
ax[0, 1].tick_params(labelrotation=45)
ax[0, 1].set_title("Series with 1% VaR Limits")
ax[0, 1].set_ylabel("Return")

# Figure Row 1 Column 3
ax[0, 2].plot(garch11_t_fit.conditional_volatility / 100.0)
ax[0, 2].set_title("Conditional SD")
ax[0, 2].tick_params(labelrotation=45)

# Figure Row 2 Column 1
sm.graphics.tsa.plot_acf(garch11_t_fit.resid / 100.0, lags=20, ax=ax[1, 0])
ax[1, 0].set_title("ACF of Observations")
ax[1, 0].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 2
sm.graphics.tsa.plot_acf(garch11_t_fit.resid**2, lags=20, ax=ax[1, 1])
ax[1, 1].set_title("ACF of Squared Observations")
ax[1, 1].set_ylim([-0.3, 0.3])

# Figure Row 2 Column 3
sm.graphics.tsa.plot_acf(np.abs(garch11_t_fit.resid), lags=20, ax=ax[1, 2])
ax[1, 2].set_title("ACF of Absolute Observations")
ax[1, 2].set_ylim([-0.3, 0.3])

# Figure Row 3 Column 1
ax[2, 0].xcorr(
    garch11_t_fit.resid**2,
    garch11_t_fit.resid,
    usevlines=True,
    maxlags=30,
    normed=True,
    lw=2,
)
ax[2, 0].set_title("Cross-Correlation of Squared Observations \n vs Actual Observat")

# Figure Row 3 Column 2
standaraized_residuals = garch11_t_fit.std_resid
min_val = np.min(standaraized_residuals)
max_val = np.max(standaraized_residuals)
empirical_density = np.linspace(min_val, max_val, len(standaraized_residuals))
ax[2, 1].plot(empirical_density, stats.norm.pdf(empirical_density), lw=1)
ax[2, 1].set_title("Empirical density of \n standardized residuals")

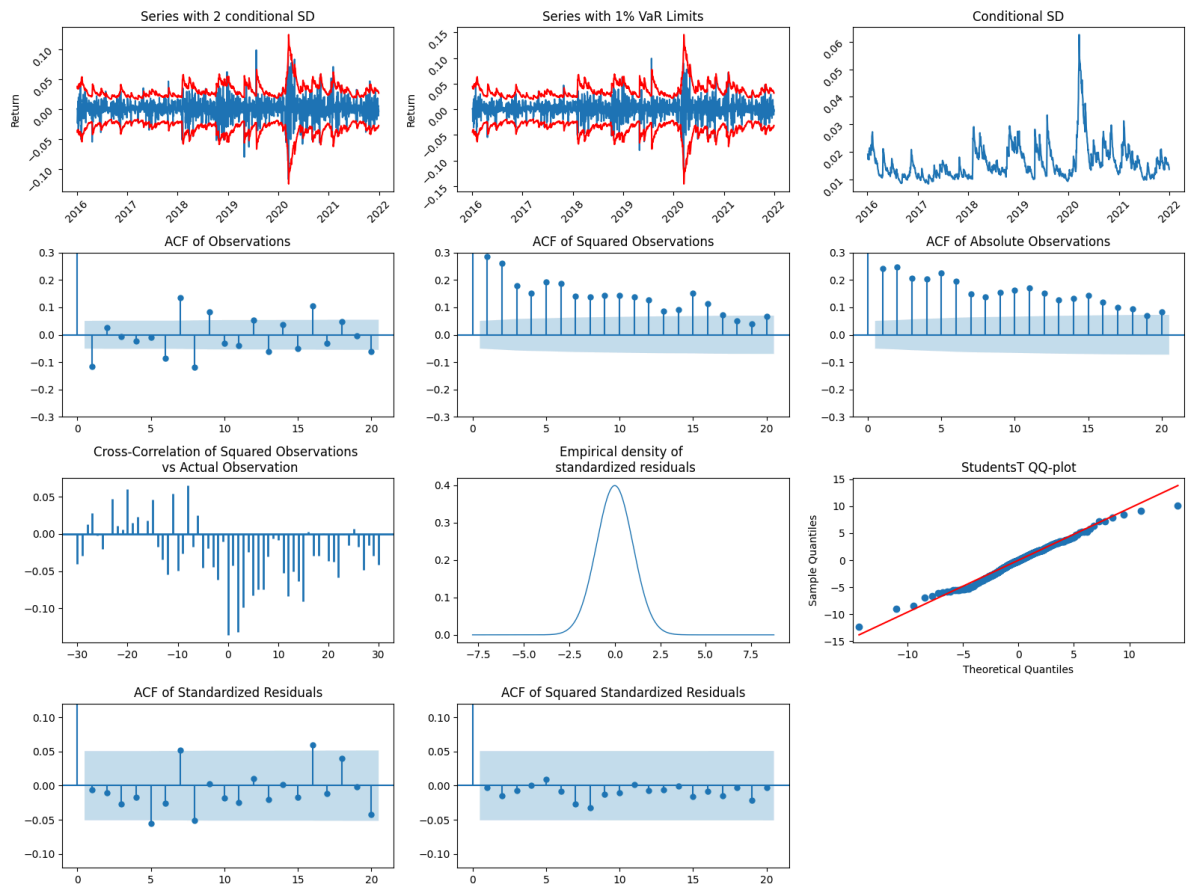
# Figure Row 3 Column 3
sm.qqplot(garch11_t_fit.resid, stats.t, fit=True, line="q", ax=ax[2, 2])
ax[2, 2].set_title("StudentsT QQ-plot")

# Figure Row 4 Column 1
sm.graphics.tsa.plot_acf(garch11_t_fit.std_resid, lags=20, ax=ax[3, 0])
ax[3, 0].set_title("ACF of Standardized Residuals")
ax[3, 0].set_ylim([-0.12, 0.12])

```

```
# Figure Row 4 Column 2
sm.graphics.tsa.plot_acf((garch11_t_fit.std_resid) ** 2, lags=20, ax=ax[3, 1])
ax[3, 1].set_title("ACF of Squared Standardized Residuals")
ax[3, 1].set_ylim([-0.12, 0.12])

ax[3, 2].axis("off")
fig.tight_layout()
plt.show()
```



From figure 9, we can see the StudentsT QQ plot shows the model is a good fit. Hence, GARCH(1,1) with Student's t white noise fits the model pretty well.

3.3 Model Forecast

Figure 10: 20 Steps Ahead Forecast for GARCH(1,1) Model with Student's t White Noise

```
In [14]: # 20 Steps Ahead Forecast for GARCH(1,1) Model with Student's t White Noise

# set horizon and forecast
horizon = 20
garch_forecast = garch11_t_fit.forecast(
    reindex=False, horizon=horizon, method="simulation"
)

# reindex data
```

```

googr = goog.GOOGLE_R.copy()
googr.index = list(range(len(googr_r) + 1))

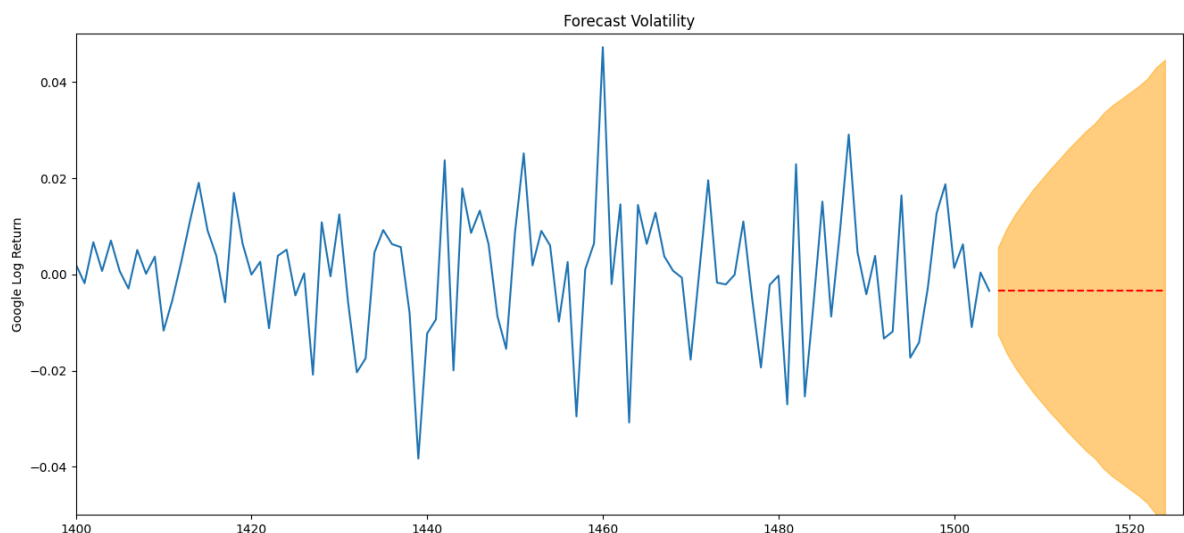
# forecast mean
forc_mean = pd.Series(garch_forecast.mean.dropna().squeeze())
forc_mean.index = list(range(len(googr), len(googr) + horizon))

# volatility forecast
variance_fct = pd.DataFrame(data={"Forecast": garch_forecast.variance.values[0]})
variance_fct.index = list(range(len(googr), len(googr) + horizon))
std_fct = [np.sqrt(variance_fct.values[i] * (i + 1)) for i in range(len(variance_fct))]
volatility_fct = pd.DataFrame(std_fct)

# upper/lower bands
upper_band = googr.values[-1] * (1.0 + 2 * volatility_fct)
upper_band.index = variance_fct.index
lower_band = googr.values[-1] * (1.0 - 2 * volatility_fct)
lower_band.index = variance_fct.index

# Plot
plt.figure(figsize=(16, 7))
plt.xlim(1400, len(googr) + 21)
plt.ylim(-0.05, 0.05)
googr.plot(ylabel="Google Log Return", title="Forecast Volatility")
plt.plot((upper_band + lower_band) / 2, "r--")
plt.fill_between(
    upper_band.index.tolist(),
    upper_band.values.T[0],
    lower_band.values.T[0],
    color="orange",
    alpha=0.5,
)
plt.show()

```



In figure 10, we plot the 20 steps ahead forecast for Google stock return data based on the GARCH(1,1) with Student's t white noise. As the forecast goes farther out, we can see that the 1-sigma bands get wider.

4. Conclusion

In this lesson, we introduced the GARCH model. We first gave the definition of GARCH(1,1) model. We then went through the properties of the GARCH(1,1) model. We also briefly expanded the GARCH model to a higher order GARCH(p, q) model. Next, we talked about estimation of the GARCH model and the tests for model diagnostics. We finished the lesson with a GARCH application to Google stock returns. In the next lesson, we will go over some details about estimation methods for the GARCH model.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Orloff, Jeremy, and Jonathan Bloom. "Bayesian Updating with Discrete Priors." Introduction to Probability and Statistics, Spring 2014, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/resources/mit18_05_s22_class11-prep_pdf/

- **Required pages:** Entire article
- **Estimated reading time:** 1 hour

2. Orloff, Jeremy, and Jonathan Bloom. "Bayesian Updating: Probabilistic Predictions." Introduction to Probability and Statistics, Spring 2014, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/resources/mit18_05_s22_class12-prep-a_pdf/

- **Required pages:** Entire article
- **Estimated reading time:** 1 hour

3. Orloff, Jeremy, and Jonathan Bloom. "Bayesian Updating with Continuous Priors." Introduction to Probability and Statistics, Spring 2014, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/resources/mit18_05_s22_class13-prep-a_pdf/

- **Required pages:** Entire article
- **Estimated reading time:** 1 hour

4. Mikusheva, Anna. Course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare, Massachusetts Institute of Technology. Downloaded on [27 July 2022]. https://ocw.mit.edu/courses/14-384-time-series-analysis-fall-2013/efc84003319dde93c94ed48d3d1e7433/MIT14_384F13_lec23and24.pdf

- **Required pages:** Entire article
- **Estimated reading time:** 1 hour

FINANCIAL ECONOMETRICS

MODULE 5 | LESSON 3

BAYESIAN ESTIMATION FOR GARCH MODEL

Reading Time 60 minutes

Prior Knowledge GARCH Model, Maximum Likelihood Estimation, Basic Statistics

Keywords Bayesian Statistics, Frequentist Statistics, Priors, Posteriors, Markov-Chain-Monte-Carlo, MCMC Convergence Diagnostics

In the last lesson, we mentioned the most commonly used estimation method for GARCH model is maximum likelihood estimation (MLE). In this lesson, we are going to introduce an alternative estimation method: Bayesian estimation. Bayesian estimation or Bayesian statistics is a branch in the field of statistics. It has a different approach from what we have learned so far to apply probability to data analysis. We will first introduce what Bayesian statistics is, how it works, and then we will discuss the differences between the Bayesian statistics and the statistical method we have been using. Then we will demonstrate how to use the Bayesian method to estimate the GARCH model. Bayesian statistics is a rich field in statistical analysis. In this lesson, we will only touch upon the surface of Bayesian statistics that should be enough for you to get through estimating a GARCH model. Apart from use in time series analysis, Bayesian statistics has gained a lot of interests in many areas that this is a subject worth diving into. We encourage interested students to seek more information about Bayesian statistics beyond this course.

In [12]: `pip install arviz pymc pytensor`

Requirement already satisfied: arviz in /usr/local/lib/python3.11/site-packages (0.21.0)

Requirement already satisfied: pymc in /usr/local/lib/python3.11/site-packages (5.22.0)

Requirement already satisfied: pytensor in /usr/local/lib/python3.11/site-packages (2.30.3)

Requirement already satisfied: setuptools>=60.0.0 in /usr/local/lib/python3.11/site-packages (from arviz) (65.6.3)

Requirement already satisfied: matplotlib>=3.5 in /usr/local/lib/python3.11/site-packages (from arviz) (3.8.2)

Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.11/site-packages (from arviz) (1.26.2)

Requirement already satisfied: scipy>=1.9.0 in /usr/local/lib/python3.11/site-packages (from arviz) (1.11.4)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/site-packages (from arviz) (23.1)

Requirement already satisfied: pandas>=1.5.0 in /usr/local/lib/python3.11/site-packages (from arviz) (2.0.0)

Requirement already satisfied: xarray>=2022.6.0 in /usr/local/lib/python3.11/site-packages (from arviz) (2023.12.0)

Requirement already satisfied: h5netcdf>=1.0.2 in /usr/local/lib/python3.11/site-packages (from arviz) (1.6.1)

Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.11/site-packages (from arviz) (4.13.2)

Requirement already satisfied: xarray-einstats>=0.3 in /usr/local/lib/python3.11/site-packages (from arviz) (0.8.0)

Requirement already satisfied: cachetools>=4.2.1 in /usr/local/lib/python3.11/site-packages (from pymc) (5.5.2)

Requirement already satisfied: cloudpickle in /usr/local/lib/python3.11/site-packages (from pymc) (3.1.1)

Requirement already satisfied: rich>=13.7.1 in /usr/local/lib/python3.11/site-packages (from pymc) (14.0.0)

Requirement already satisfied: threadpoolctl<4.0.0,>=3.1.0 in /usr/local/lib/python3.11/site-packages (from pymc) (3.2.0)

Requirement already satisfied: filelock>=3.15 in /usr/local/lib/python3.11/site-packages (from pytensor) (3.18.0)

Requirement already satisfied: etuples in /usr/local/lib/python3.11/site-packages (from pytensor) (0.3.9)

Requirement already satisfied: logical-unification in /usr/local/lib/python3.11/site-packages (from pytensor) (0.4.6)

Requirement already satisfied: miniKanren in /usr/local/lib/python3.11/site-packages (from pytensor) (1.0.3)

Requirement already satisfied: cons in /usr/local/lib/python3.11/site-packages (from pytensor) (0.4.6)

Requirement already satisfied: h5py in /usr/local/lib/python3.11/site-packages (from h5netcdf>=1.0.2->arviz) (3.13.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->arviz) (1.2.0)

Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->arviz) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->arviz) (4.46.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->arviz) (1.4.5)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->arviz) (9.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->arviz) (3.1.1)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->arviz) (2.8.2)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.5.0->arviz) (2023.3.post1)
 Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.5.0->arviz) (2023.3)
 Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/site-packages (from rich>=13.7.1->pymc) (3.0.0)
 Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/site-packages (from rich>=13.7.1->pymc) (2.15.1)
 Requirement already satisfied: toolz in /usr/local/lib/python3.11/site-packages (from logical-unification->pytensor) (1.0.0)
 Requirement already satisfied: multipledispatch in /usr/local/lib/python3.11/site-packages (from logical-unification->pytensor) (1.0.0)
 Requirement already satisfied: mdurl<=0.1 in /usr/local/lib/python3.11/site-packages (from markdown-it-py>=2.2.0->rich>=13.7.1->pymc) (0.1.2)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.7->matplotlib>=3.5->arviz) (1.16.0)
 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip available: 22.3.1 -> 25.2

[notice] To update, run: `pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

```
In [13]: import arviz as az
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pymc as pm
from pymc import GARCH11
from pytensor import shared
import pytensor
pytensor.config.mode = 'NUMBA'
```

```
In [14]: # Download the dataset
m5_data = pd.read_csv("M5. goog_eur_10.csv")

# Convert date variable to date format
m5_data["Date2"] = pd.to_datetime(m5_data["Date"], format="%m/%d/%Y")
goog = m5_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
goog["GOOGLE_R"] = np.log(goog.GOOGLE).diff().dropna()
```

1. Bayesian Statistics

1.1 What is Bayesian Statistics?

Bayesian statistics uses Bayes' theorem as the foundation to apply probabilities to data analysis. From the method we have been learning so far, we assume parameters like mean, variance, and standard deviations are unknown but fixed parameters. This method is called

frequentist statistics. Maximum likelihood estimation belongs to frequentist statistics. On the other hand, in Bayesian statistics, mean, variance and standard deviations are random variables with probability distributions. This is the main difference between frequentist statistics and Bayesian statistics. To run Bayesian statistics, a researcher has to either use experience or other techniques to assume a parameter's probability distribution before running the analysis. We call these probability distributions **prior probability distributions** or **priors**. We then combine the information from data to derive **posterior probability distributions for parameters** or **posteriors**. The required readings in this lesson will provide definitions and applications of these concepts in more detail. It is good that now the parameters are not unknown fixed values but rather random variables. This change can make analysis more flexible. However, this method sometimes is criticized for subjective parameter probability distribution choice by the researcher before running the data analysis. There is a lot of debate about advantages and disadvantages of frequentist statistics vs. Bayesian statistics. For students who are interested in knowing more detail, there is a lot of material online.

1.2 Basics of Bayesian Updating, Prior Probability Distributions, Posterior Probability Distributions, and Likelihood Function

Before continuing this lesson, please read the required readings for this lesson. The required readings will cover the concept of Bayesian updating, prior probability distributions (priors), posterior probability distributions (posteriors), and likelihood functions in detail. In the following sections of this lesson, we will use these concepts extensively so it is important that you are familiar with these concepts before proceeding to the next section.

1.3 Sampling from Posterior Probability Distribution

We learned in the last section how to derive posterior probability distributions for parameters we are interested in analyzing. We know a posterior distribution is the multiplication of the likelihood function and the prior distribution. Oftentimes, a posterior distribution formula is too complicated to manipulate or is in high dimension. Hence, we have to conduct sampling to obtain a series of sample data points from the posterior distribution to investigate its marginal distribution density, mean, and other parameters.

The most popular sampling method is the **Markov-Chain-Monte-Carlo (MCMC)** method. Monte Carlo is a random sampling method when the samples are independent. Markov-Chains address random samples that are autocorrelated to each other. Hence, MCMC method is a random sampling method for autocorrelated samples. There are several sampling algorithms under the MCMC method. The two most popular ones are the **Metropolis-Hastings algorithm** and the **Gibbs Sampling algorithm**. For those who are interested in the details of these algorithms, there are plenty of materials to explore online.

For the following sections, the software we are using to conduct Bayesian estimation on GARCH models uses the Metropolis-Hastings algorithm.

One final note about sampling before we move on to Bayesian estimation of GARCH models: The MCMC method can be sensitive to the initial values used to start sampling. Therefore, sometimes, we would use the result from maximum likelihood estimation as the starting values for Bayesian estimation. We will demonstrate this point when we show how to use Bayesian estimation in a later section.

2. Bayesian Estimation on GARCH Model

In this section, we are going to go over the steps to use Bayesian method to fit a GARCH model. Specifically, we will focus on a GARCH model with Student's t innovations (errors). From the last lesson, we see that one common feature of financial asset returns is their heavy tail distribution. Hence, it is reasonable to use Student's t-distribution to model asset returns.

Ardia and Hoogerheide (2010) demonstrated the steps using Bayesian estimation to fit a GARCH model with Student's t innovations. They wrote an R package (bayesGARCH) to automate the sampling process by using the Metropolis-Hastings algorithm. In the next section, we will use our Google stock return as an example to show the process. Before diving into the example, let's go over the theory for GARCH model with Student's t innovations first.

$$r_t = \epsilon_t \sqrt{\frac{\nu - 2}{\nu} \omega_t \sigma_t^2}$$

where

$$\begin{aligned} t &= 1, \dots, T \\ \epsilon_t &\sim \text{iid normal}(0, 1) \\ \omega_t &\sim \text{iid inverse gamma}\left(\frac{\nu}{2}, \frac{\nu}{2}\right) \\ \sigma_t^2 &= \alpha_0 + \alpha_1 r_{t-1}^2 + \beta \sigma_{t-1}^2 \text{ where } \alpha_0 > 0, \alpha_1, \beta \geq 0 \text{ and } \nu > 2. \end{aligned}$$

The restrictions above will ensure that the variance will be positive and finite. However, they do not ensure stationarity. We know from the last lesson that $\alpha_1 + \beta < 1$ is needed to ensure stationarity. We will use the result in our application later.

Another point we would like to explain is the inclusion of ω_t . The reason we have ω_t as a prior multiplied with the variance of r_t in the model is that the marginal distribution of r_t will become a Student's t-distribution with degrees of freedom ν .

The priors for $\alpha_0, \alpha_1, \beta$ are assumed to be truncated normal distributions with the domains greater than 0 since these restrictions ensure positive and finite variance. The prior for ν is shifted exponential distribution.

The joint posterior distribution will look like:

$$p(\alpha_0, \alpha_1, \beta, \nu, \omega_t | r_t) = \frac{l(r_t | \alpha_0, \alpha_1, \beta, \nu, \omega_t) p(\alpha_0, \alpha_1, \beta, \nu, \omega_t)}{p(r_t)}$$

Then, MCMC will sample from the joint posterior distribution for each parameter. From the sampled data, we can generate the density for each parameter. As we mentioned in the last section, MCMC sampling can be sensitive to initial values. One method to avoid extreme values from initial values is to drop the first n numbers of sampled observations. This is called a **burn-in** period. We can indicate to the package how many initial sampled observations we would like to drop from our sample series. We will elaborate more on this topic in the next section.

In this section, we will use Google stock return to fit a GARCH model by using the Bayesian estimation method. We will use Python package: PyMC to run this model. The PyMC package has inbuilt GARCH(1,1) distribution that will make the code easier to implement. We should bear in mind that the inbuilt GARCH distribution is with Normal innovations. However, from the Ardia and Hoogerheide (2006) we can learn that the priors for Bayesian GARCH(1,1) with Normal innovations remain the same. Hence, we provide this example for demonstration purposes.

We will use MCMC to generate two data sample series with 3000 samples in each series. In MCMC, a sample series is called a **chain**. Why do we need to sample two data series? We generate two chains because we need to use them to check if the sample series are close enough to the posterior that we want to analyze. This is called **convergence diagnostics** for MCMC sampling. There are several methods to conduct convergence diagnostics. We will use the Gelman-Rubin Diagnostic. The diagnostic provides a "Scale Reduction Factor" to indicate if several series converge. Usually, a value below 1.2 is okay. Since this diagnostic requires at least two chains, we will create two chains from MCMC. Figure 1 shows the MCMC sampling result for the joint posterior distribution for Google stock return GARCH model.

Figure 1: First Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

```
In [15]: # First Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plot

# starting parameters = blank canvas model(0.000001, 0.000001, 0.000001)
alpha_mu = shared(np.array([0.000001, 0.000001], dtype=np.float64))
alpha_sigma = shared(np.array([[1000.0, 0.0], [0.0, 1000.0]], dtype=np.float64))

beta_mu = shared(np.array(0.000001, dtype=np.float64))
beta_sigma = shared(np.array(1000.0, dtype=np.float64))

# initial volatility
ivolatility = shared(np.array(0.000001, dtype=np.float64))
ivolatility_vol = shared(np.array(10.0, dtype=np.float64))
```

```

# construct MCMC model
mcmc0 = pm.Model()

with mcmc0:
    # create multivariate normal:  $p(\alpha) \propto \phi_{N2}(\alpha/\mu\alpha, \Sigma\alpha) 1\{\alpha \in R^{2+}\}$ 
    mvn = pm.MvNormal("mvNormal", mu=alpha_mu, cov=alpha_sigma, shape=2)

    # restrict the two coefficients to be greater than zero
    alp0 = pm.Deterministic("alpha0", pm.math.switch(mvn[0] > 0, mvn[0], -np.inf))
    alp1 = pm.Deterministic("alpha1", pm.math.switch(mvn[1] > 0, mvn[1], -np.inf))

    # create univariate truncated normal:  $p(\beta) \propto \phi_{N1}(\beta/\mu\beta, \Sigma\beta) 1\{\beta \in R+\}$ 
    nTruncated = pm.TruncatedNormal("beta", mu=beta_mu, sigma=beta_sigma, lower=0)
    volTruncated = pm.TruncatedNormal(
        "volatility", mu=ivolatility, sigma=ivolatility_vol, lower=0
    )

    # Likelihood
    likelihood = GARCH11(
        "GARCH",
        omega=alp0,
        alpha_1=alp1,
        beta_1=nTruncated,
        initial_vol=volTruncated,
        observed=goog.GOOGLE_R.dropna() * 100,
    )

```

```

In [16]: # Plot first round MCMC model posteriors
with mcmc0:
    step_mcmc0 = pm.Slice()
    trace_mcmc0 = pm.sample(
        3000,
        cores=2,
        step=step_mcmc0,
        tune=0, # tune=0 => no burn-in
        return_inferencedata=True,
        random_seed=12345,
    )
az.plot_trace(
    trace_mcmc0,
    var_names=["alpha0", "alpha1", "beta"],
    # plot vertical lines at GARCH(1,1) MLE param results
    lines=[
        ("alpha0", {}, [0.124993]),
        ("alpha1", {}, [0.082160]),
        ("beta", {}, [0.867127]),
    ],
    compact=False,
    legend=True,
    figsize=(16, 7),
)
plt.tight_layout()
plt.show()

```

```
Multiprocess sampling (2 chains in 2 jobs)
```

```
CompoundStep
```

```
>Slice: [mvNormal]
```

```
>Slice: [beta]
```

```
>Slice: [volatility]
```

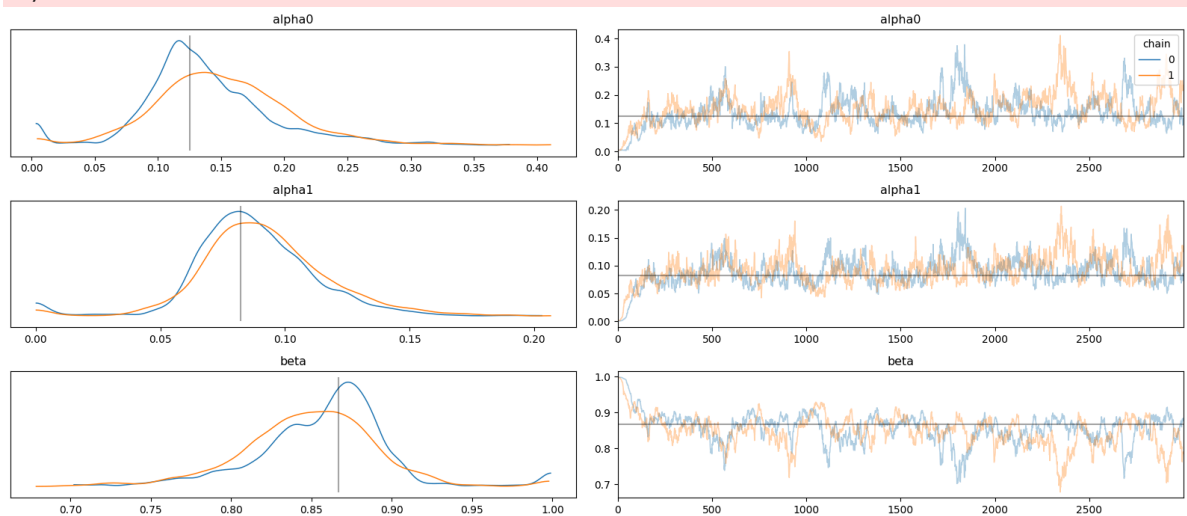
```
Output()
```

Sampling 2 chains for 0 tune and 3_000 draw iterations (0 + 6_000 draws total) took 105 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

The rhat statistic is larger than 1.01 for some parameters. This indicates problems during sampling. See <https://arxiv.org/abs/1903.08008> for details

The effective sample size per chain is smaller than 100 for some parameters. A higher number is needed for reliable rhat and ess computation. See <https://arxiv.org/abs/1903.08008> for details



On the right of figure 1 are the plots called trace plots for parameters. Let's focus on the trace plot of *alpha0* for now to learn how to read the plot. There are two series on the trace plot. One is blue and the other is orange. They represent the two chains sampled from MCMC. Let's focus on the blue one first. The trace plot lays out the way that the first sampled data is on the leftmost side of the plot, and the second sampled data follows it to the right and so on and so forth. We can see that the first 300 samples start with very volatile movement. As more and more sampled data is collected, the series starts to converge to a value of around 0.12. However, there is some volatility between the 1700th sample and 3000th sample. The orange chain in general demonstrates a pattern similar to the blue chain. Let's run the summary test to see how well the MCMC sample converges.

Figure 2: First Round MCMC Convergence Test Result and Parameter Estimations

```
In [17]: # First Round MCMC Convergence Test Result and Parameter Estimations
az.summary(trace_mcmc0, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

Out[17]:

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail
alpha0	0.145094	0.053294	0.059351	0.262754	0.007721	0.004627	47.599936	51.832998
alpha1	0.089880	0.025170	0.048496	0.141094	0.003876	0.002370	47.796746	48.588985
beta	0.852806	0.042358	0.757265	0.916286	0.006640	0.004178	44.814853	48.402712

Last column in Figure 2 for r_hat represents the Gelman-Rubin statistic. It tests for lack of convergence by comparing the variance between multiple chains to the variance within each chain. If convergence has been achieved, the between-chain and within-chain variances should be identical. If $r_hat < 1.2$ for all model parameters, we can have some confidence that convergence has been reached.

Let's look at the estimations for the parameters. Figure 2 shows the estimated parameters from the first round MCMC sampling result. Let's focus on the means of the coefficients. We see $alpha0$, $alpha1$ and $beta$ all look reasonable within our restrictions for stationary and positive and finite variance. But so far the model represents so called 'blank canvas' model i.e. we have not accounted for any meaningful starting values. In this initial model the starting values are all near zero for all $alpha0$, $alpha1$ and $beta$ priors.

We discussed in the previous section that the MCMC can be sensitive to starting value. If starting value is way off, it can impact the final result of MCMC. In this case, we can assign starting values to parameters before the MCMC starts sampling. In that way, we restrain MCMC from going wild when sampling. So where can we find initial values for parameters? In the previous lesson, we estimated the parameters using maximum likelihood. We can use these estimates as starting values for MCMC. Otherwise, this step is a trial-and-error process. You might need to run MCMC a number of times for MCMC to converge to a solution and find reasonable coefficient estimates.

For the next round of MCMC, we adjust starting values for parameters in the code and rerun MCMC. The following figures show the final results of the second run MCMC sampling.

Figure 3: Second Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

In [18]:

```
# Second Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

# starting parameters = MLE(0.124993, 0.082160, 0.867127)
alpha_mu = shared(np.array([0.124993, 0.082160], dtype=np.float64))
alpha_sigma = shared(np.array([[1000.0, 0.0], [0.0, 1000.0]], dtype=np.float64))

beta_mu = shared(np.array(0.867127, dtype=np.float64))
beta_sigma = shared(np.array(1000.0, dtype=np.float64))

# initial volatility
ivolatility = shared(
    np.array(1.63865, dtype=np.float64))
```

```

) # np.std(goog["GOOGLE_R"].dropna()*100)
ivolatility_vol = shared(np.array(10.0, dtype=np.float64))

# construct MCMC model
mcmc = pm.Model()

with mcmc:
    # create a multivariate normal:  $p(\alpha) \propto \phi_{N2}(\alpha/\mu\alpha, \Sigma\alpha) 1\{\alpha \in R^{2+}\}$ 
    mvn = pm.MvNormal("mvNormal", mu=alpha_mu, cov=alpha_sigma, shape=2)

    # restrict the two coefficients to be greater than zero
    alp0 = pm.Deterministic("alpha0", pm.math.switch(mvn[0] > 0, mvn[0], -np.inf))
    alp1 = pm.Deterministic("alpha1", pm.math.switch(mvn[1] > 0, mvn[1], -np.inf))

    # create univariate truncated normal:  $p(\beta) \propto \phi_{N1}(\beta/\mu\beta, \Sigma\beta) 1\{\beta \in R+\}$ 
    nTruncated = pm.TruncatedNormal("beta", mu=beta_mu, sigma=beta_sigma, lower=0)
    volTruncated = pm.TruncatedNormal(
        "volatility", mu=ivolatility, sigma=ivolatility_vol, lower=0
    )

    # likelihood
    likelihood = GARCH11(
        "garch",
        omega=alp0,
        alpha_1=alp1,
        beta_1=nTruncated,
        initial_vol=volTruncated,
        observed=goog.GOOGLE_R.dropna() * 100,
    )

```

In [19]: # Plot Second round MCMC model posteriors

```

with mcmc:
    step_mmc = pm.Slice()
    trace_mmc = pm.sample(
        3000,
        cores=2,
        step=step_mmc,
        tune=0, # no burn-in
        return_inferencedata=True,
        random_seed=12345,
    )
    az.plot_trace(
        trace_mmc,
        var_names=["alpha0", "alpha1", "beta"],
        lines=[
            ("alpha0", {}, [0.124993]),
            ("alpha1", {}, [0.082160]),
            ("beta", {}, [0.867127]),
        ],
        compact=False,
        legend=True,
        figsize=(16, 7),
    )
    plt.tight_layout()
    plt.show()

```

```

Multiprocess sampling (2 chains in 2 jobs)
CompoundStep
>Slice: [mvNormal]
>Slice: [beta]
>Slice: [volatility]
Output()

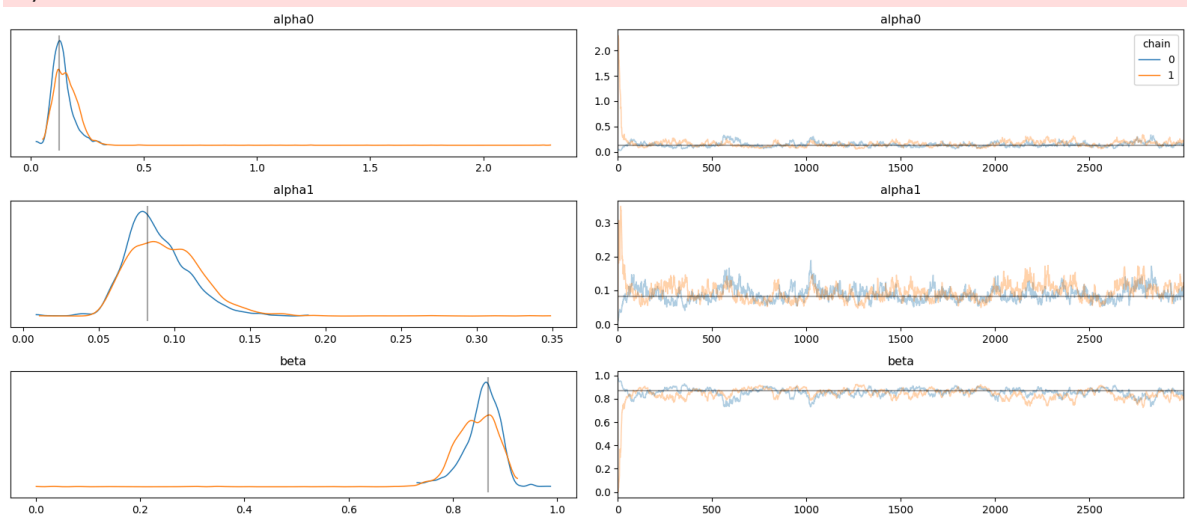
```

Sampling 2 chains for 0 tune and 3_000 draw iterations (0 + 6_000 draws total) took 102 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

The rhat statistic is larger than 1.01 for some parameters. This indicates problems during sampling. See <https://arxiv.org/abs/1903.08008> for details

The effective sample size per chain is smaller than 100 for some parameters. A higher number is needed for reliable rhat and ess computation. See <https://arxiv.org/abs/1903.08008> for details



In figure 3, we can see that after adding starting values to the code, the trace plot for all priors converges better in this round of MCMC. Let's check the Gelman-Rubin convergence test and other parameter estimations.

Figure 4: Second Round MCMC Convergence Test Result

```

In [20]: # Second Round MCMC Sampling Result: Parameter estimation
az.summary(trace_mmc, round_to=7).loc[["alpha0", "alpha1", "beta"], :]

```

```

Out[20]:

```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_ta
alpha0	0.151440	0.091908	0.067844	0.240691	0.007156	0.043919	82.938764	221.22757
alpha1	0.092149	0.024706	0.054129	0.132585	0.002455	0.002861	92.183544	318.26632
beta	0.848298	0.051242	0.782080	0.913977	0.004892	0.015678	75.874103	231.98836



We can see from figure 4 that all the parameters have converged pretty well after adding starting values to the code and r_hat improved.

However, we can still see from figure 3 that the first 250 data points seem to be very volatile. We can treat them as the burn-in period and drop those sampled data points. At this point, we will also combine chains. Let's check out the MCMC result after these steps.

Figure 5: Second Round MCMC Sampling Result: Parameter Trace Plots and Density Plots After Adjustment

```
In [ ]: # Second Round MCMC Sampling Result: Parameter Trace Plots and Density Plots After
with mcmc:
    step_mmc_a = pm.Slice()
    trace_mmc_a = pm.sample(
        3000,
        cores=2,
        step=step_mmc_a,
        tune=250, # burn-in = 250 steps
        return_inferencedata=True,
        random_seed=12345,
    )
    az.plot_trace(
        trace_mmc_a,
        var_names=["alpha0", "alpha1", "beta"],
        lines=[
            ("alpha0", {}, [0.124993]),
            ("alpha1", {}, [0.082160]),
            ("beta", {}, [0.867127]),
        ],
        compact=False,
        legend=True,
        figsize=(16, 7),
    )
    plt.tight_layout()
    plt.show()
```

```
Multiprocess sampling (2 chains in 2 jobs)
CompoundStep
>Slice: [mvNormal]
>Slice: [beta]
>Slice: [volatility]
Output()
```

Sampling 2 chains for 250 tune and 3_000 draw iterations (500 + 6_000 draws total) took 86 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

The effective sample size per chain is smaller than 100 for some parameters. A higher number is needed for reliable rhat and ess computation. See <https://arxiv.org/abs/1903.08008> for details

From figure 5, we can see the trace plots for 3 parameters all oscillate within a narrower range compared to the trace plots in figure 1. On the left, we can see that the marginal densities of *alpha0* and *alpha1* seem to skew to the right while the marginal density for *beta* skew to the left. Let's check the final parameter estimates.

Figure 6: First Round MCMC Sampling Result: Parameter Estimation with Adjustment

```
In [ ]: # First Round MCMC Sampling Result: Parameter Estimation with Adjustment
az.summary(trace_mmc_a, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

In figure 6, we can see that all estimated coefficients are well behaved and confined in the restrictions to meet stationarity and positive and finite variance requirements. In figure 7 below, we can compare the estimated coefficients from MLE and Bayesian estimations. We can see the estimated coefficients are similar. We achieved this comparable result partly because we used MLE estimates as our starting values for MCMC sampling. By carefully selecting starting values for MCMC, we can control the behavior of the estimated coefficients.

Figure 7: Google Stock Return, GARCH Model Estimated Parameters Comparison Between MLE and Bayesian

	MLE(norm)	MLE(st)	Bayesian
omega/alpha0	0.1250	0.0616	0.1436
alpha1	0.0822	0.1039	0.0901
beta	0.8671	0.8865	0.8530

4. Conclusion

In this lesson, we first introduced Bayesian statistics. We discussed Bayesian statistics and the main difference between the frequentist and Bayesian methods. Then, we looked at Bayesian updating, prior distributions, likelihood functions, and posterior distributions. Next, we described how MCMC can sample data from posterior distribution for analysis. Finally, we use the Google stock return to run a GARCH model and described the steps required to run Bayesian estimation for this GARCH model. Bayesian statistics is a powerful tool to learn apart from the frequentist method we are already very familiar with. Bayesian statistics will also play an important role in machine learning, which you will learn in the following course.

References

- Ardia, David, "Bayesian Estimation of the GARCH(1,1) Model with Normal Innovations", Munich Personal RePEc Archive, September 2006
 - Ardia, David, and Lennart F. Hoogerheide. "Bayesian Estimation of the GARCH(1,1) Model with Student-t Innovations." The R Journal, vol. 2, 2010, pp. 41–47.
-

Copyright 2025 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Mikusheva, Anna. Course materials for 14.384 Time Series Analysis, Fall 2007. MIT OpenCourseWare, Massachusetts Institute of Technology. Downloaded on [27 July 2022]. https://ocw.mit.edu/courses/14-384-time-series-analysis-fall-2013/3dcb917d20ece78852cd398a3f7b5e64/MIT14_384F13_lec21.pdf

- **Required pages:** Entire lecture
- **Estimated reading time:** 1 hour 30 minutes

FINANCIAL ECONOMETRICS

MODULE 5 | LESSON 4

GARCH MODEL UNDER STATE SPACE MODEL CONSTRUCT

Reading Time	60 minutes
Prior Knowledge	Time Series analysis, GARCH
Keywords	State Space Model, Markov Process, Kalman Filter

In this lesson, we will use a state space approach to create a GARCH model. The state space model (SSM) was originally developed as an analytical method for aerospace-related research. However, it has gained a lot of interest in engineering and finance recently. We will give a brief description of the structure of state space model and its properties. We will then introduce a popular linear state space model: Kalman Filter. We will go through the basics of the Kalman Filter. Lastly, we will combine Kalman Filter with GARCH(1,1) model to give you insight into how the combined model works.

1. State Space Models

1.1. The Basics of a State Space Model

State space model (or dynamic linear model, DLM) is another popular time series model class. State space model (SSM) states that there are two types of time series variables: one that we can observe and one that we cannot observe. The ones that we cannot observe are called **states**. Those that we can observe are called **observations** instead. The states are generated by a system that describes how states evolve over time. We would like to find out the structure of this system. However, we only see observations. Observations are dependent on states with noise added. We use an observation equation to describe the relationship between observations and states. We also add a measurement noise to the observation equation to fill the gap between the observations and states. We can formalize the above concept in the following equations. Assume y_t is the time series observation variable and x_t is the time series state variable:

Observation Equation:

$$y_t = A_t x_t + v_t, \quad v_t \sim WN(0, V_t) \tag{1}$$

where v_t is the measurement noise.

State Equation:

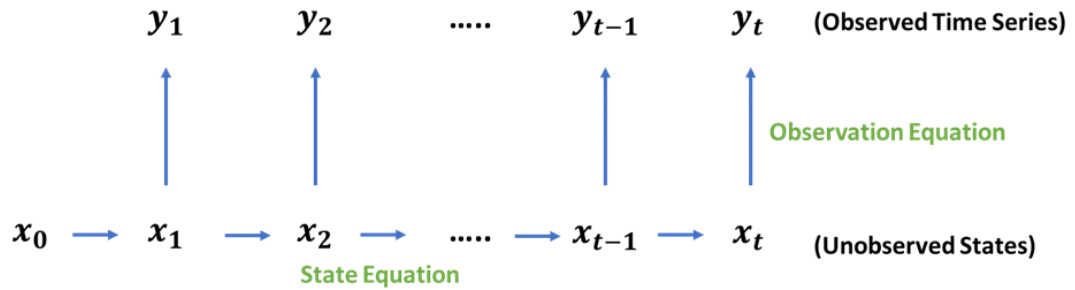
$$x_t = \Phi_t x_{t-1} + w_t, \quad w_t \sim WN(0, W_t) \quad (2)$$

where w_t is the system noise.

We also assume that v_t and v_s are independent when $t \neq s$, w_t and w_s are independent when $t \neq s$, and v_t and w_s are uncorrelated on all lags.

The above two equations form the basis of a state space model. We can also use the following graph to demonstrate the structure of a state space model.

Figure 1: Relationship between States and Observations in a State Space Model



From figure 1, we can see the evolution of the state variable is represented by a state equation. This is the system or the model created by theory or process. State equation(s) can be a single equation or a set of equations to describe how states are generated. Then, we have the observation equation to connect states from a state equation to observations with a measurement noise. As for the number of state variables and observation variables, there can be more than one state variable or observation variable in a state space model. The number of state variables also do not have to be the same as the number of observation variables. As for state space model design, it can be linear or non-linear. In equation (1) and equation (2), we can see that A_t and Φ_t both have time subscript. It means they can change by time. If this is the case, it is a time-varying state space model. If A_t and Φ_t are fixed regardless of time (no time subscript), it is a time-invariant state space model.

We can see state space model is a very general and flexible time series modeling format. You can construct a time series model for state equation and then add an observation equation to connect states with observations or data.

Once we build a state space model, we will use the model to achieve the following three goals:

- a. Predicting:** forecast future values of states given all the information and data we have today
- b. Filtering:** estimate current values of states given all the information and

data we have today

c. Smoothing: estimate past values of states given all the information and data we have today

1.2. Properties of State Space Model

Now let's look at two key properties from the state space model.

a. x_t is a Markov Process

It means $P(x_t | x_{t-1}, x_{t-2}, \dots, x_0) = P(x_t | x_{t-1})$. Markov process states that the probability of the value of a state at current time only depends on the value of the most recent state.

The values of the state from older times do not have an impact on today's value. We can say that x_t is memoryless.

b. y_t is fully specified given x_t

This statement can be shown as follows:

$$P(y_t | x_t, x_{t-1}, \dots, x_0; y_{t-1}, y_{t-2}, \dots, y_1) = P(y_t | x_t).$$

With the above two properties, we can write the joint distribution function for states and observations as follows:

$$P(x_{0:t}, y_{1:t}) = P(x_0) \prod_{i=1}^t P(x_i | x_{i-1}) P(y_i | x_i)$$

For x_0 , we usually have to assume its value or its distribution in order to proceed with the model building.

1.3 Benefits of State Space Model (SSM)

There are several reasons why state space modeling is gaining popularity among researchers. Firstly, according to the properties we explained above, we only need the most recent information to estimate the value of the current state. This property will reduce the amount of historical data needed and data storage space required to estimate SSM.

Secondly, as we saw above, SSM allows for time varying coefficients. It is very helpful if the modeling period happens to have regime change. Time varying SSM is good for capturing this change.

Other advantages of SSM include:

- a. Use SSM to handle missing values by using forecast function.
- b. Apply SSM to build a model for complex multivariate systems.
- c. Use SSM to build a model for non-stationary time series.

In the next section, we will introduce the Kalman filter. The Kalman filter is the most studied state space model, and we will use Kalman filter as the base state space model to build a GARCH(1,1) model.

1.4. Estimation of Parameters for State Space Model

Once the state space model is specified, we can estimate the coefficients. We have to estimate these parameters: A_t , Φ_t , V_t and W_t .

The most popular estimation method is MLE. However, Bayesian estimation is also another viable choice for parameter estimation. Actually, Bayesian estimation is a suitable approach for a state space model given that the nature of the question of SSM is to estimate the values of states given all the observations we know. In fact, we will use a Bayesian approach to discuss our next topic: Kalman filter.

2. Kalman Filter

2.1. Kalman Filter Model Setup

Kalman filter is a linear state space model with normal (Gaussian) noise. It has been very popular due to its recursive method that can avoid heavy computing and storage issues happening to other statistical analysis methods. In this section, we will study a time invariant multivariate Kalman filter model. The properties of the Kalman filter derived in this section still apply to Kalman filter models where some or all of the parameters are time varying. Let's write down the model and discuss its properties.

Assume we have x_t as a $r \times 1$ state vector and y_t as a $n \times 1$ observation vector. The model can be written as follows:

Observation Equation: $y_t = F'x_t + v_t$, v_t is the measurement noise.

State Equation: $x_t = Gx_{t-1} + w_t$, w_t is the system noise.

where $v_t \sim N(0, V)$ and $w_t \sim N(0, W)$.

We also assume the initial value of x_t to be x_0 and x_0 has the $x_0 \sim N(m_0, C_0)$ distribution and m_0 and C_0 are known.

Following the assumption from the last section, we still assume that v_t and v_s are independent when $t \neq s$, w_t and w_s are independent when $t \neq s$, and v_t and w_s are uncorrelated on all lags.

In the model, F is a $r \times n$ matrix, and it is called an observation matrix or design matrix. G is a $r \times r$ matrix and it is called a state transition matrix.

Given so many variables we have mentioned so far, we will summarize these variables in the following list:

- x_t : $r \times 1$ state vector at time t ;
- y_t : $n \times 1$ observation vector at time t ;
- F : $r \times n$ observation matrix between observation and state at time t ;
- G : $r \times r$ state transition matrix between current state at t and prior state at $t - 1$;
- v_t : measurement noise from multivariate normal distribution;
- w_t : system noise from multivariate normal distribution;
- V : variance-covariance matrix for multivariate normal distribution where measurement noise is from;
- W : variance-covariance matrix for multivariate normal distribution where system noise is from;
- m_0 : mean value of multivariate normal distribution from initial state, x_0 ;
- C_0 : variance-covariance matrix of multivariate normal distribution from initial state, x_0 .

2.2. Kalman Filter Model Derivation

In the last section, we went through the Kalman filter model setup and described all the variables in this model. In this section, we are going to explain how Kalman filter model works and what the key results are from this model. We are going to use a Bayesian approach to explain how a Kalman filter model works.

In the previous section, we mentioned that the concept of using all the information from the past and new data to infer the unobserved state in a state space model is very similar to the Bayesian updating process. Let's try to demonstrate what a state space model is looking for in terms of a Bayes theorem.

For a Kalman filter model, we can express it using the following Bayes theorem formula.

$$P(x_t|y_t, D_{t-1}) = \frac{P(y_t|x_t)P(x_t|D_{t-1})}{P(y_t)}$$

Where x_t is current state, y_t is current observation, D_{t-1} is all the pass information, including pass states and pass observations.

Here is the breakdown of prior probability, likelihood function, and posterior probability.

- Posterior probability: $P(x_t|y_t, D_{t-1})$
- Likelihood function: $P(y_t|x_t)$
- Prior probability: $P(x_t|D_{t-1})$

The above equation says the probability of getting the value of state x_t at time t , given we know the current observation y_t and all our prior knowledge D_{t-1} , is the multiplication of the likelihood of y_t happening given x_t and the probability of the state x_t happening given all our prior knowledge D_{t-1} . We also normalize the multiplication by dividing the probability of y_t happening. This is the process of updating the value of current state x_t with the new information y_t .

With the above Bayesian equation and some mathematical manipulations, we can derive the following results:

Prior: $x_t|D_{t-1} = x_t|y_{1:t-1} \sim N(a_t, R_t)$

Where $a_t = Gm_{t-1}$ and $R_t = G'C_{t-1}G + W$

Likelihood: $y_t|x_t \sim N(Fx_t, V)$

One-Step Forecast: $y_t|y_{t-1} \sim N(f_t, Q_t)$

Where $f_t = Fa_t$ and $Q_t = F'R_tF + V$

Posterior: $x_t|D_t = x_t|y_{1:t} \sim N(m_t, C_t)$

Where $m_t = a_t + K_te_t$

$C_t = R_t - K'_tQ_tK_t = (1 - K_tF)R_t$

$K_t = R_tFQ_t^{-1}$ (K_t is called Kalman gain matrix)

$e_t = y_t - f_t$ and $e_t \sim N(0, Q_t)$

The above posterior result is called Kalman filter recursion. Remember there are three goals for the state space model described before. This result is to achieve the filtering goal: Use all the past information and new data to estimate the current value of the state. We use the mean of the posterior distribution m_t as our estimated value of the current state x_t , given all the information we have.

Let's look at the above result intuitively. By looking at the mean of the posterior equation, we can see m_t is a linear combination of previous state m_{t-1} and new information y_t . We can view the estimated value of the current state as the weighted average of the value of the past state and new information. K_t (Kalman gain matrix) is one of the weights in m_t formula. We can see K_t as a tuning factor to decide if more weight should be put on new information or not. The higher the value of K_t , the more weight the model puts on new information. Also, the estimated current state is only dependent on the state value of the last period and new value; there is no need to have all the historical data to update the current state. This recursive method of updating current state value with new information and only the last period of the state makes the forecast of Kalman filter a lot easier than other methods.

2.3 Forecasting with Kalman Filter

One key reason to learn Kalman filter is to use it to conduct forecasting. Forecasting includes predicting x_{t+d} and y_{t+d} given $y_{1:t}$. We can apply the result from the last section to conduct forecasting.

When $d = 1$, the forecasting result can be derived from prior and one-step forecast equations from last section.

State at $t + 1$:

$x_{t+1} = Gm_t$ forecast state

$R_{t+1} = G'C_tG + W$ forecast variance of state

Observation at $t + 1$:

$y_{t+1} = Fx_{t+1}$ forecast observation

$Q_{t+1} = F'R_{t+1}F + V$ forecast variance of observation

When $d > 1$, we can use a recursive method to obtain the result as follows:

State at $t + d$:

$x_{t+d} = Gm_{t+d-1}$ forecast state

$R_{t+d} = G'C_{t+d-1}G + W$ forecast variance of state

Observation at $t + d$:

$y_{t+d} = Fx_{t+d}$ forecast observation

$Q_{t+d} = F'R_{t+d}F + V$ forecast variance of observation

3. Use Kalman Filter to Fit GARCH(1,1) Model

In this section, we are going to apply Kalman filter to build a GARCH(1,1) model. We have studied in the previous lessons that GARCH model has conditional heteroskedasticity feature on the observation level. Hence, we need to incorporate this feature to a Kalman filter model. We can write the model as follows:

$$y_t = Fx_t + z_t$$

$$x_t = Gx_{t-1} + w_t, \text{ where } w_t \sim N(0, W)$$

$$z_t = s_t\epsilon_t, \text{ where } \epsilon_t \sim N(0, 1)$$

$$s_t^2 = \alpha_0 + \alpha_1 z_{t-1}^2 + \beta_1 s_{t-1}^2$$

The first two equations from the above equation system are from the Kalman filter. The first one is the observation equation and the second one is the state equation. In the observation equation, we can see the measurement noise has a further specification from the third and fourth equations. The third and fourth equations are the section for GARCH. The measurement noise is now following standard normal GARCH(1,1) set up.

From the above system of equations for Kalman filter-GARCH(1,1) model, we can do some manipulation and obtain the posterior and one-step forecast result.

Posterior for state: $x_t|y_{1:t} \sim N(m_t, C_t)$

Where all the parameters are the same as standard Kalman filter model

One-step forecast for observation: $y_t|y_{t-1} \sim N(f_t, Q_t)$

Where $f_t = Fa_t$ and $Q_t = F'R_tF + s_t^2$

We can see from the above result, the only difference between the standard Kalman filter model and Kalman filter-GARCH(1,1) model is the s_t^2 in Q_t , the variance for observation.

Once we have the posterior and one-step forecast equations set up, we can use either Gibbs sampling or Metropolis-Hasting sampling method to sample unknown parameters. For unknown state vectors, we use a technique called forward filtering and backward sampling (FFBS) to sample. FFBS was proposed by Carter and Kohn as well as Fruhwirth-Schnatter. Most statistical software will provide FFBS sampling function for use. For more information on how to sample, Omar provides detailed steps to sample prior distribution and simulate data with FFBS on Kalman filter GARCH(1,1) model.

4. Shiny Application For GARCH(1,1) Model

In this module, we provided a Shiny application to build a GARCH(1,1) model on an asset return.

[Click here to access the application.](#) First, you choose an asset return for the model. There are three choices: Google stock price daily return, EUR/USD daily return, and U.S. 10-year Treasury bond yield daily return. You then use the error distribution for a GARCH(1,1) model. There are four choices: normal, Student's t, skew normal, and skew Student's t.

The data summary tab will show the time series plot, histogram, and QQ plot of the selected asset return. The squared return summary tab will show the time series plot for the squared return. This tab will also provide ACF and PACF plots for both daily returns and squared daily returns of the selected asset return. The last tab will present the GARCH(1,1) model result and model diagnostic information.

5. Conclusion

In this lesson, we went through the theory of state space model. We talked about the basics of state space model, its properties, its benefits, and its estimation method. We then discussed a popular linear state space model, Kalman Filter. We explained how Kalman Filter works, how to derive the model and how to use it to forecast. After learning Kalman Filter, we combined Kalman Filter with GARCH(1,1) model and gave a brief model setup for Kalman Filter-GARCH(1,1) model. This lesson concludes our study on conditional heteroskedasticity

in time series data. In the next module, we will dive into the method to check for stationary for a time series. We will also look into the relationship of two time series.

References

- Carter, Chris, and Robert Kohn. "On Gibbs Sampling for State Space Models." *Biometrika*, vol. 81, 1994, pp. 541–553.
- Fruhwirth-Schnatter, Sylvia. "Data Augmentation and Dynamic Linear Models." *Journal of Time Series Analysis*, vol. 15, 1994, pp. 183–202.
- Omar, Zayd. *State-Space Models with GARCH Errors: Application to Health Data*. 2019. McGill University, Master's thesis.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.